



大學模型技術 實踐大工程術 語言應用實務班

講師：陳仁政 Ph.D

clement1972@gmail.com

<https://sites.google.com/site/clement1972/>



陳仁政

白熊 / Ph.D.

AI 應用工程師 LLM 系統

開發 技術教育工作者

學術背景

- **博士學位**

中原大學電子研究所

- **博士後研究**

中央研究院資訊科學研究所

現職

實踐大學

推廣中心 講師

緯育 TibaMe

LLM 實務課程 講師

工研院產業學院

AI 應用課程 講師

福建理工大學

計算機科學與數學學院 外聘教師

過去經歷

冠捷科技

正工程師

104 人力銀行人資學院

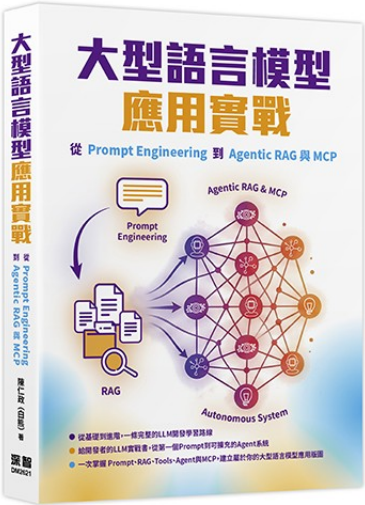
資料科學家

長庚大學工商管理系

兼任實務教師

講師著作

課程教課書



深智數位 | DM2621
2026 年 03 月出版

大型語言模型應用實戰

從 Prompt Engineering 到 Agentic RAG 與 MCP

- Prompt Engineering
- RAG 系統
- Tools / Function Call
- MCP
- Agentic RAG

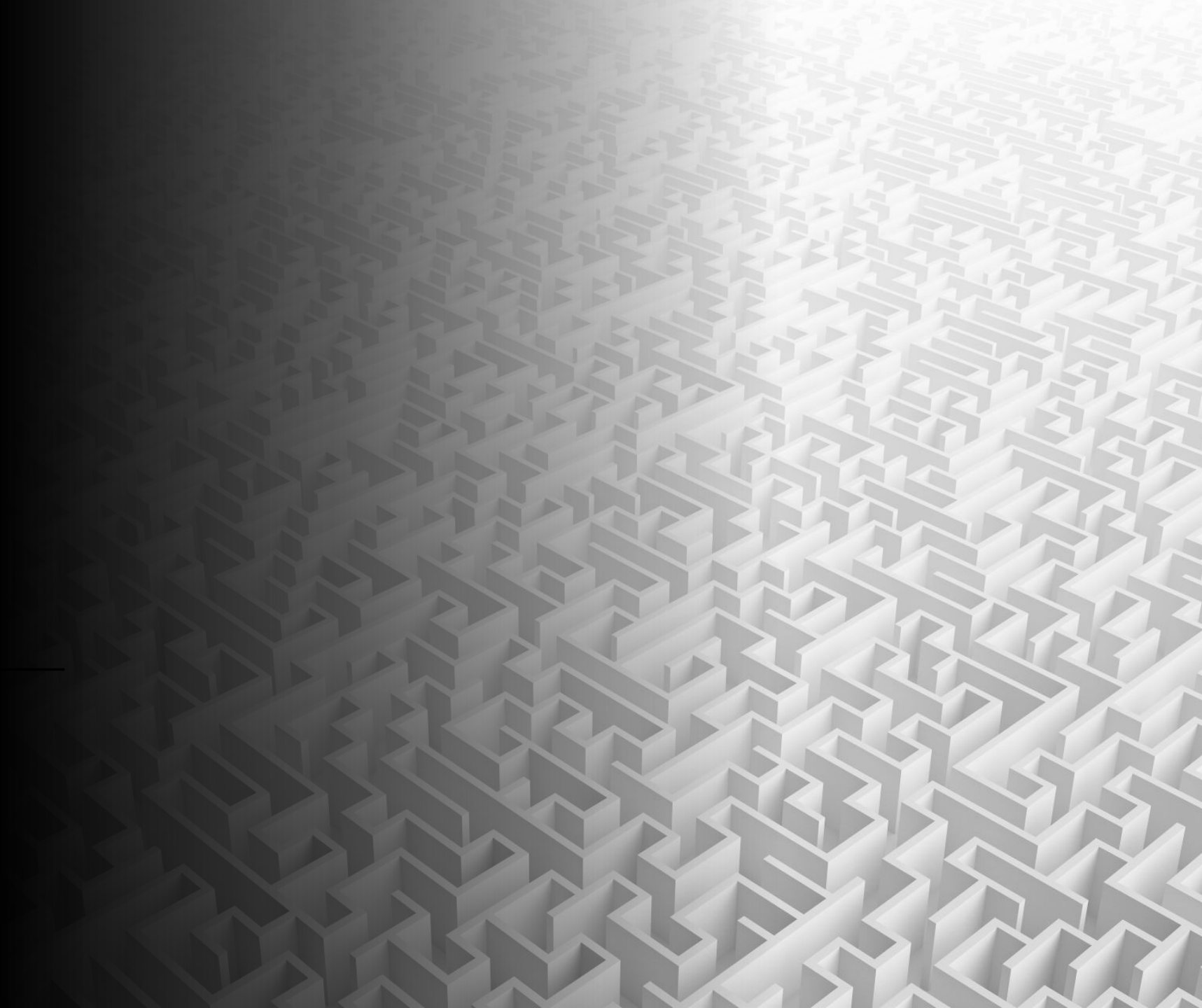
ISBN : 978-626-375-789-5 424 頁 / 繁體中文

課綱大綱

- LLM 基本控制
- 提示詞工程
- RAG
- Tools
- MCP
- Advance RAG



LLM 原理



LLM 概念

大型語言模型 □ 文字接龍產生器

“機器學習的”



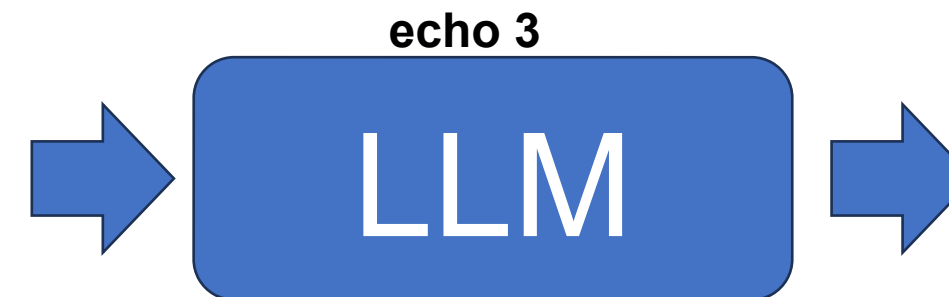
“定”

“機器學習的定”



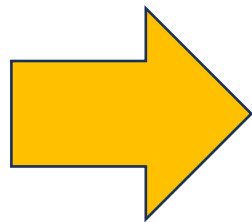
“義”

“機器學習的定義”



“是”

LLM



**如何製造這樣的
機器？**

統計法：

定 ☒ 出現 7000 次
好 ☒ 出現 500 次
原 ☒ 出現 2500 次

LLM

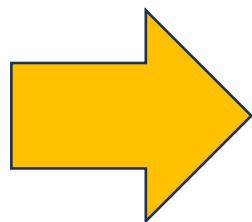


“機器學習的” ☒ 所有文章出現過 10000 次

統計法 (N-Gram Language Models)

- 常用在輸入法提示
- 無法處理長距離依賴
 - 由於僅考慮固定數量的前詞，N-Gram 模型難以捕捉句子中較遠詞語之間的依賴關係。
- 資料稀疏性問題
 - 隨著 N 的增加，可能出現的 N-Gram 組合數量急劇上升，導致許多組合在訓練語料中未出現，影響模型的準確性。
- 無法處理語法和語義信息
 - N-Gram 模型僅考慮詞語的順序，未能充分利用語法和語義信息，限制了其在複雜語言任務中的表現。

LLM



**使用機器學習演
算法**

機器學習

計算型智慧

機器學習是一種讓電腦能夠從資料中學習，並根據經驗進行預測或判斷的技術，而不需要被明確地編程來執行特定任務。

人工智慧

符號人工智慧

1950s~1980s

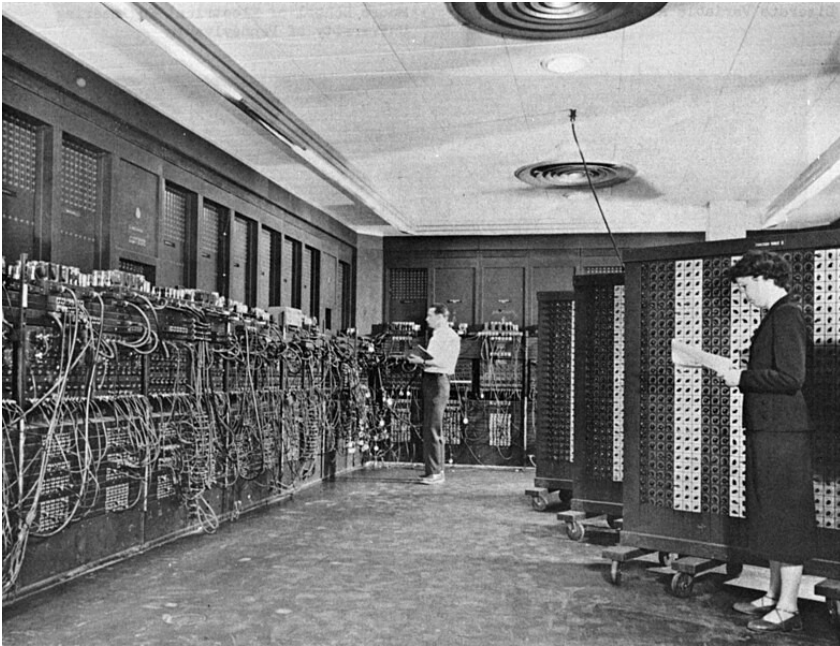
機器學習

1980~ 迄今

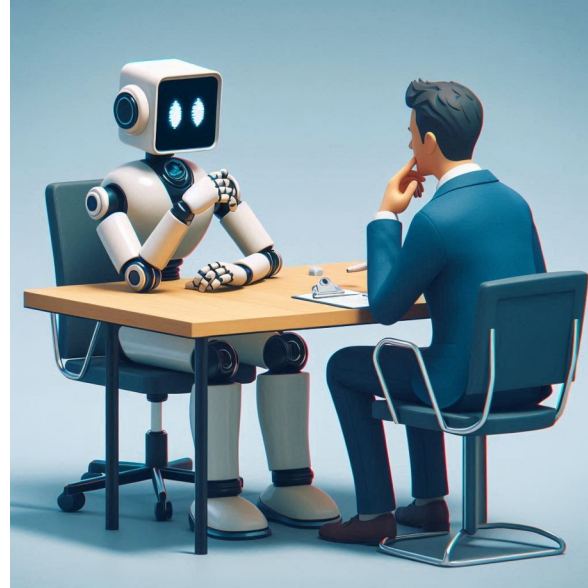
深度學習

2012~ 迄今

1946 ENIAC



1950 圖靈測



1956 達特茅斯會

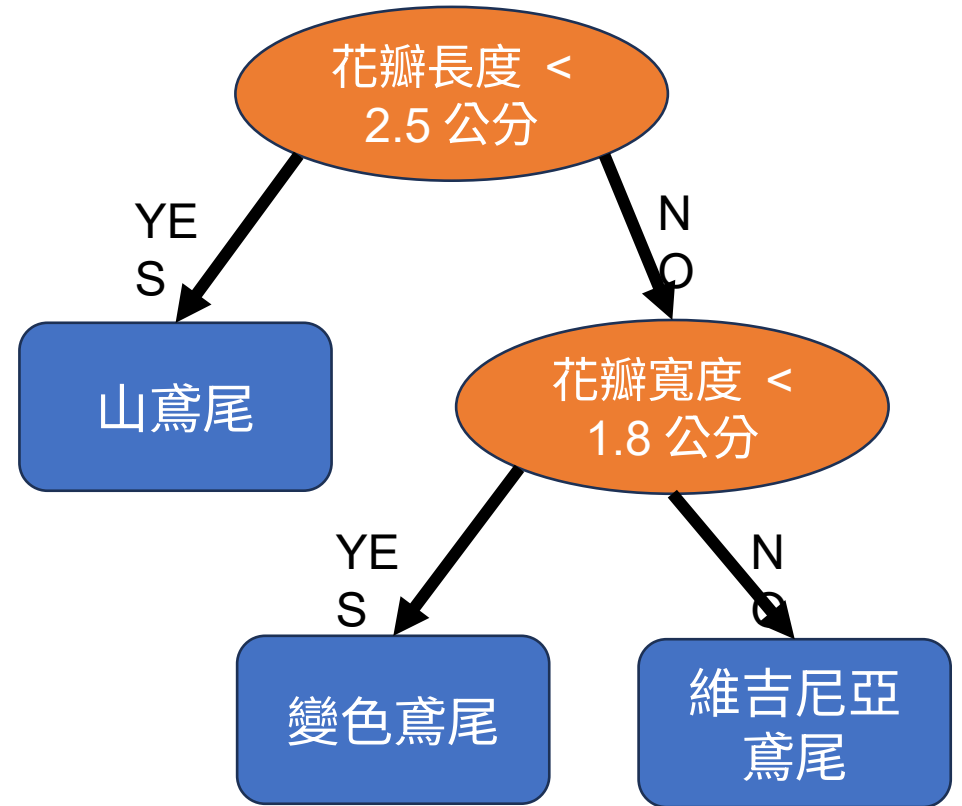


符號人工智慧 Symbolic artificial intelligence

- 1950 年代中期到 1980 年代後期
- 專家系統：基於規則和知識庫進行推理
 - MYCIN：醫療診斷 (1970 年代)
 - Exsys：第一個成功商業化的專家系統 (1983)

符號人工智慧 Symbolic AI

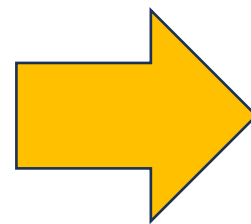
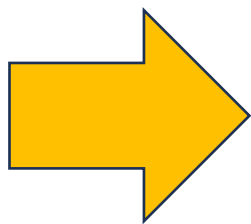
- 使用規則當成知識庫
 - 以鳶尾花分類為例
 - 如果花瓣長度小於 2.5 公分，則是山鳶尾
 - 如果花瓣長度大於 2.5 公分 且 花瓣寬度小於 1.8 公分，則是變色鳶尾
 - 如果花瓣長度大於 2.5 公分 且 花瓣寬度大於 1.8 公分，則是維吉尼亞鳶尾
- 規則由領域專家與 AI 工程師合作建立
- 常見專家系統語言：Prolog 、 Lisp



機器學習 Machine Learning

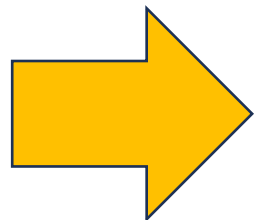
- 計算型智慧
 - 數值計算（數據驅動）
- 1980~ 迄今
- 常見演算法
 - K 近鄰算法 (KNN)
 - 決策樹、隨機森林
 - 主成分分析 (PCA)、線性回歸、支持向量機 (SVM)
 - 類神經網路 (Neural Network)

X
特徵欄位

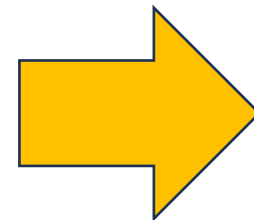


y
預測結果

圖片
留言文字
房屋資訊
語音
基因資料



機器學習
Machine
Learning



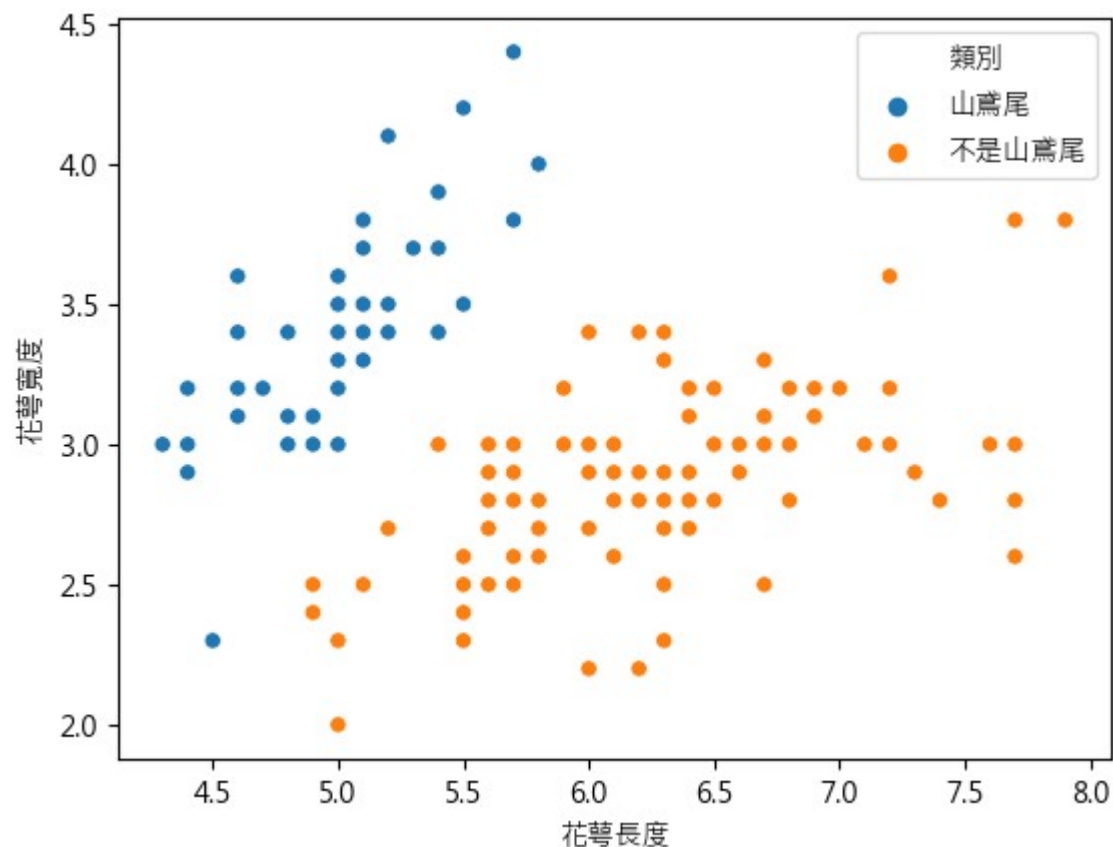
人臉位置
情緒分析
房價
文字
疾病預測

機器學習訓練資料範例

花萼長度	花萼寬度	類別
5.1	3.5	山鳶尾
4.9	3	山鳶尾
4.7	3.2	山鳶尾
7	3.2	不是山鳶尾
6.4	3.2	不是山鳶尾
6.9	3.1	不是山鳶尾
6.3	3.3	不是山鳶尾
5.8	2.7	不是山鳶尾
7.1	3	不是山鳶尾

特徵欄位 x

預測目標 y



$$y = f(\text{花萼長度}, \text{花萼寬度})$$



$$w1 * \text{花萼長度} + w2 * \text{花萼寬度} > b$$



$$-3.4 * \text{花萼長度} + 3.15 * \text{花萼寬度} > -8.44$$

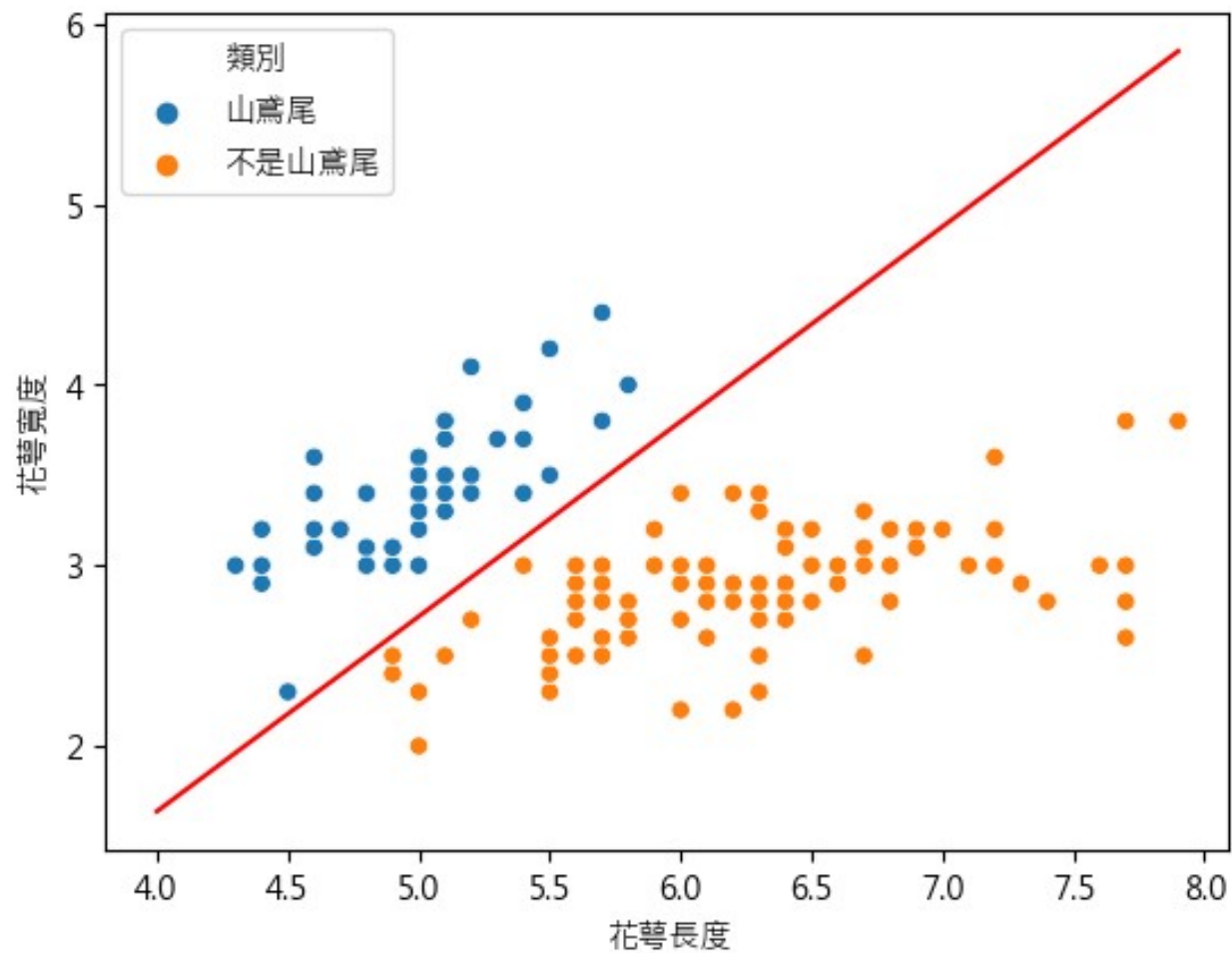
測試方程式的效果

$-3.4 \times \text{花萼長度} + 3.15 \times \text{花萼寬度} + 8.44 > 0$

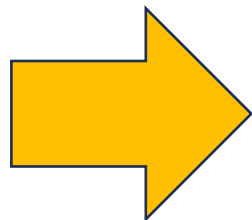
花萼長度	花萼寬度	類別	
5.1	3.5	山鳶尾	$-3.4 \times 5.1 + 3.15 \times 3.5 + 8.44 = 2.125$
4.9	3	山鳶尾	$-3.4 \times 4.9 + 3.15 \times 3 + 8.44 = 1.23$
4.7	3.2	山鳶尾	
7	3.2	不是山鳶尾	$-3.4 \times 7 + 3.15 \times 3.2 + 8.44 = -5.28$
6.4	3.2	不是山鳶尾	
6.9	3.1	不是山鳶尾	
6.3	3.3	不是山鳶尾	
5.8	2.7	不是山鳶尾	$-3.4 \times 5.8 + 3.15 \times 2.7 + 8.44 = -2.77$
7.1	3	不是山鳶尾	

紅線

$$-3.4 * \text{花萼長度} + 3.15 * \text{花萼寬度} + 8.44 = 0$$



機器學習



權重怎麼來？

線性回歸 Linear Regression

$$y = w_1x_1 + w_2x_2 + \cdots + w_nx_n + w_0b$$

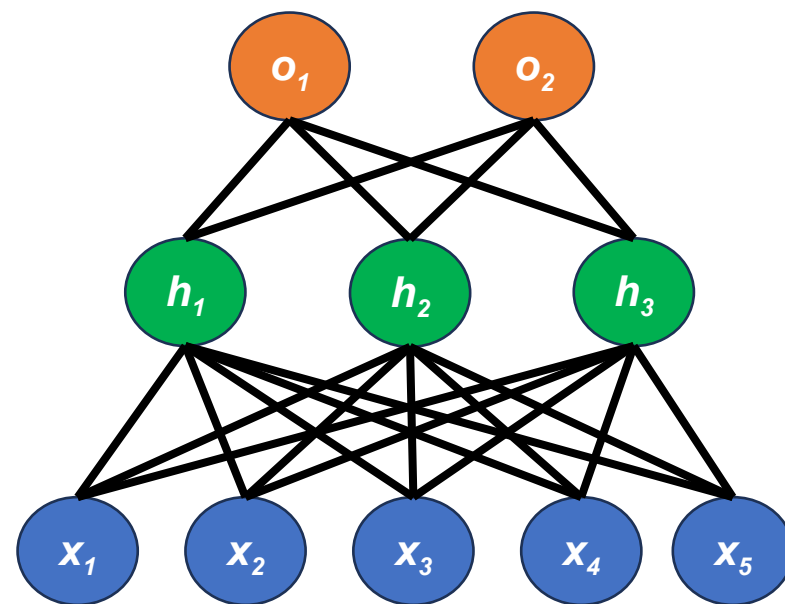
使用矩陣表示： $y = \mathbf{X}\mathbf{w}$

線性回歸是

使用推導出的方程式來一步就計算出權重 $\mathbf{w}$

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Neural Network



類神經網路的基本概念

神經元

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

使用矩陣表示：

$$y = f(\mathbf{W} \cdot \mathbf{x} + \mathbf{b})$$

輸入 (x)

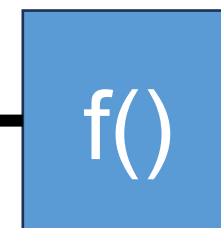
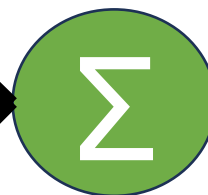
x_1

x_2

x_3

...

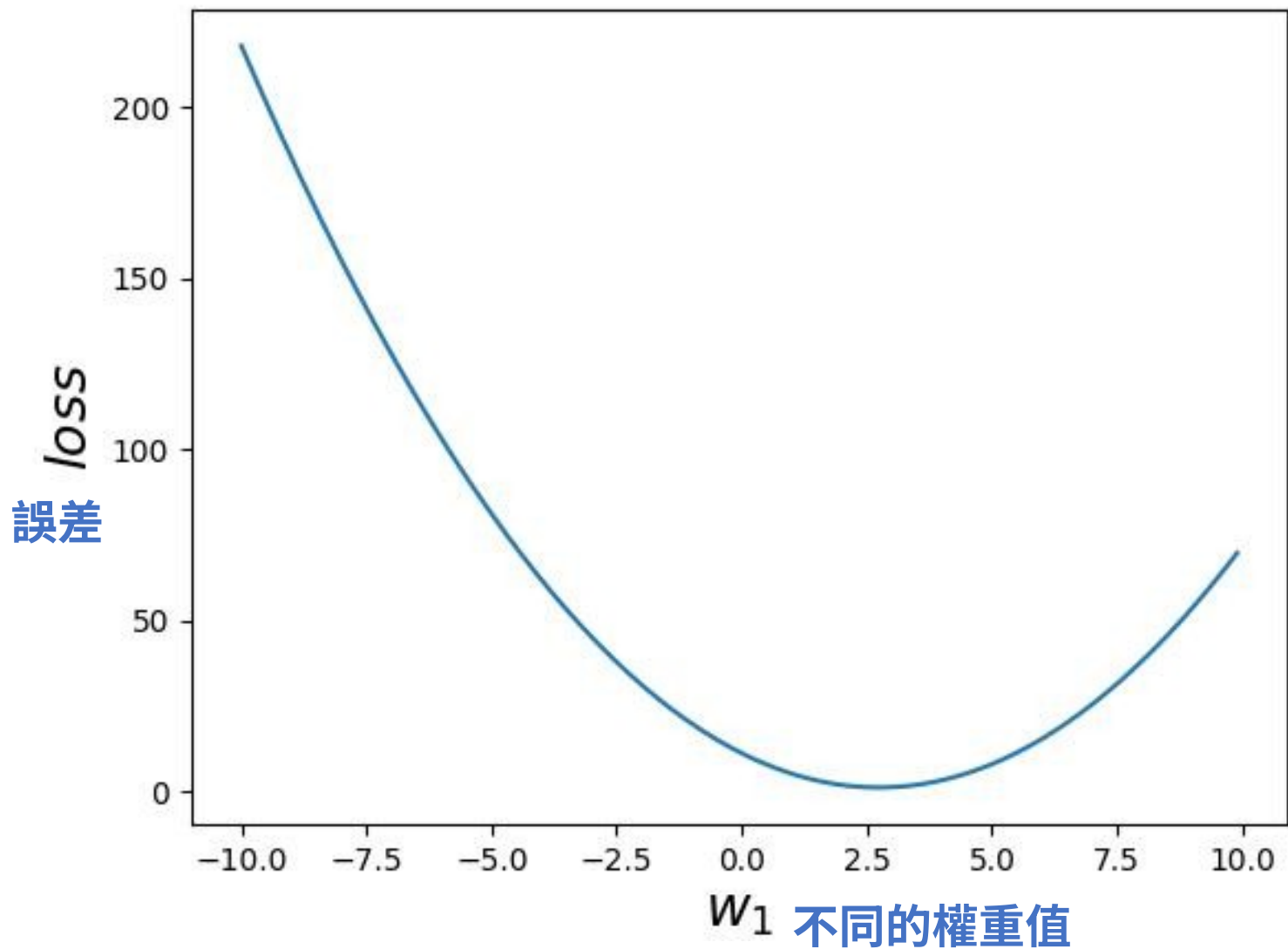
x_n



輸出 (y)

y

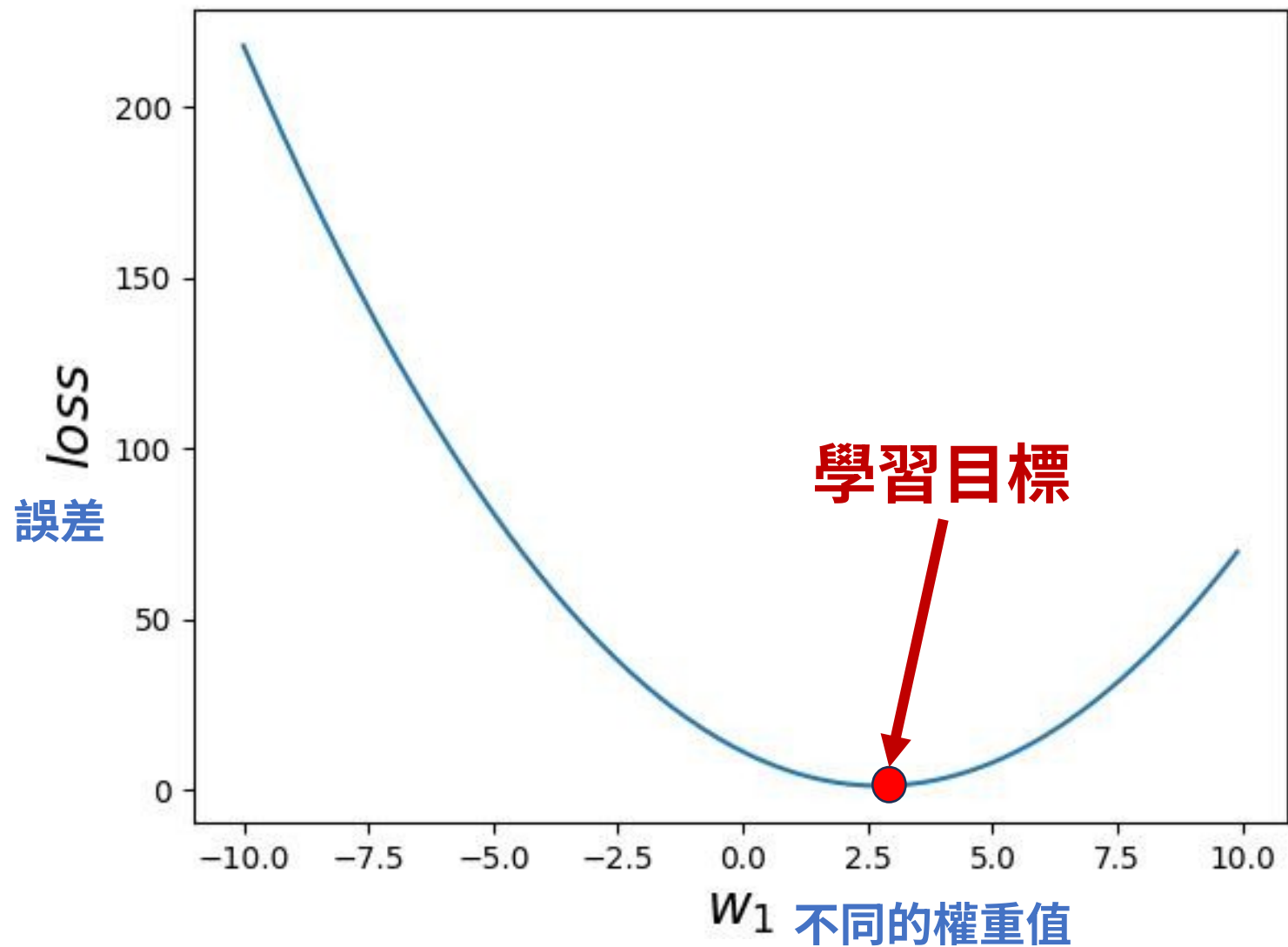
權重 (weight)



類神經網路模型的訓練可以看成是一個找正確權重值的過程。

隨著神經網路模型的權重值的不同，預測的正確性也會有差異。這個差異習慣上會使用誤差 (loss) 來表示。

如果我們把所有可能的權重值都窮舉，並把造成的誤差都計算過一遍。可以畫成左邊的圖。



類神經網路訓練的目標：
找出誤差最小的地方，但是
不能用窮舉法

Gradient Descent 梯度下降法

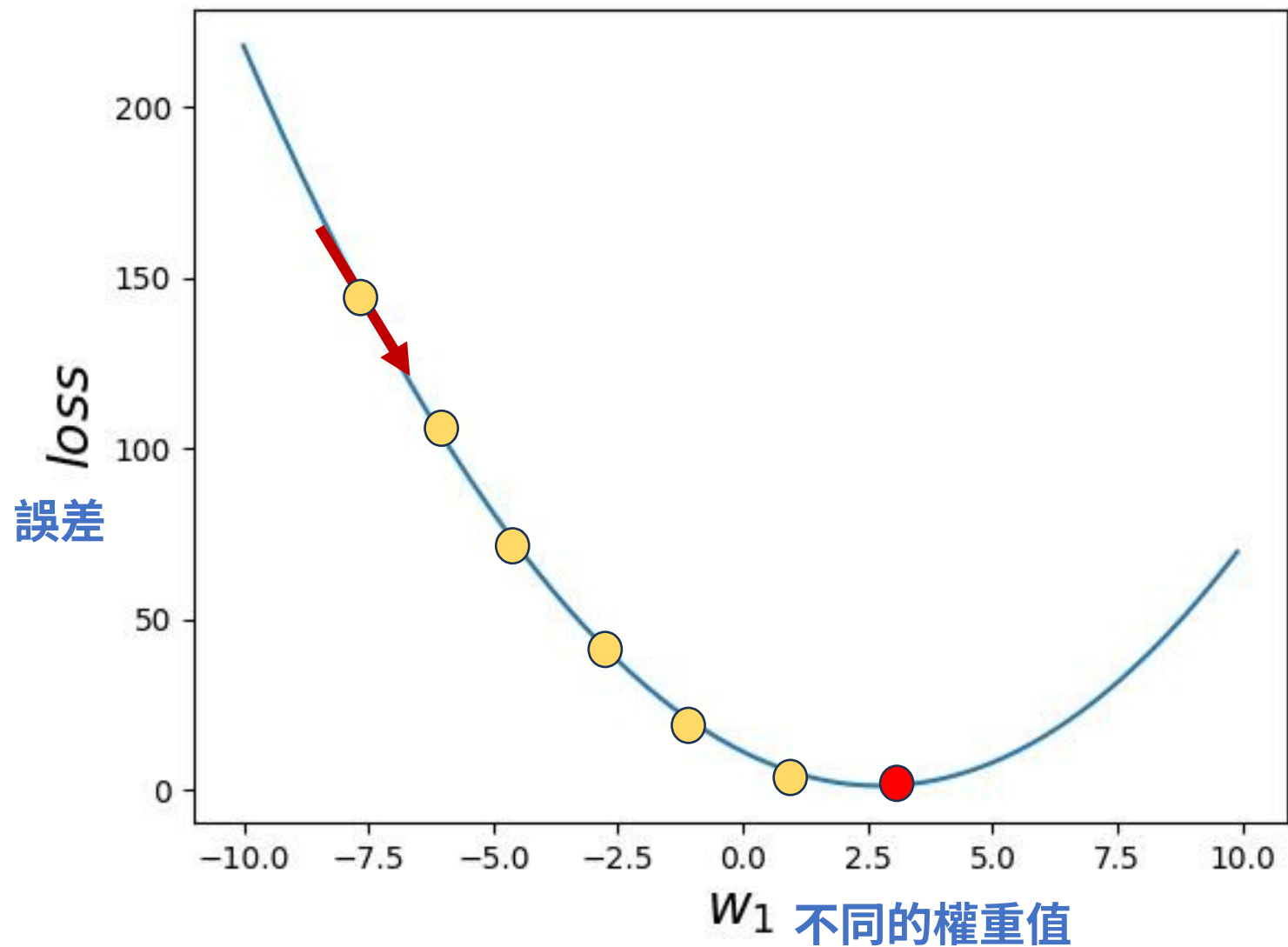
梯度下降法就像是在山上摸黑往山谷走，我們每次都往「地勢比較低」的方向走一小步，目標是走到最低點（也就是最佳解）。

怎麼走？

- 1.先站在某個起點（初始參數）
- 2.看看地勢的斜率（梯度）
- 3.往「下坡」方向走一小步：

 新的位置 = 現在的位置 - 學習率 × 斜率

Gradient Descent 梯度下降法



一開始先隨機設定權重值，
譬如 $w = -7.5$ ，然後計算
出誤差。

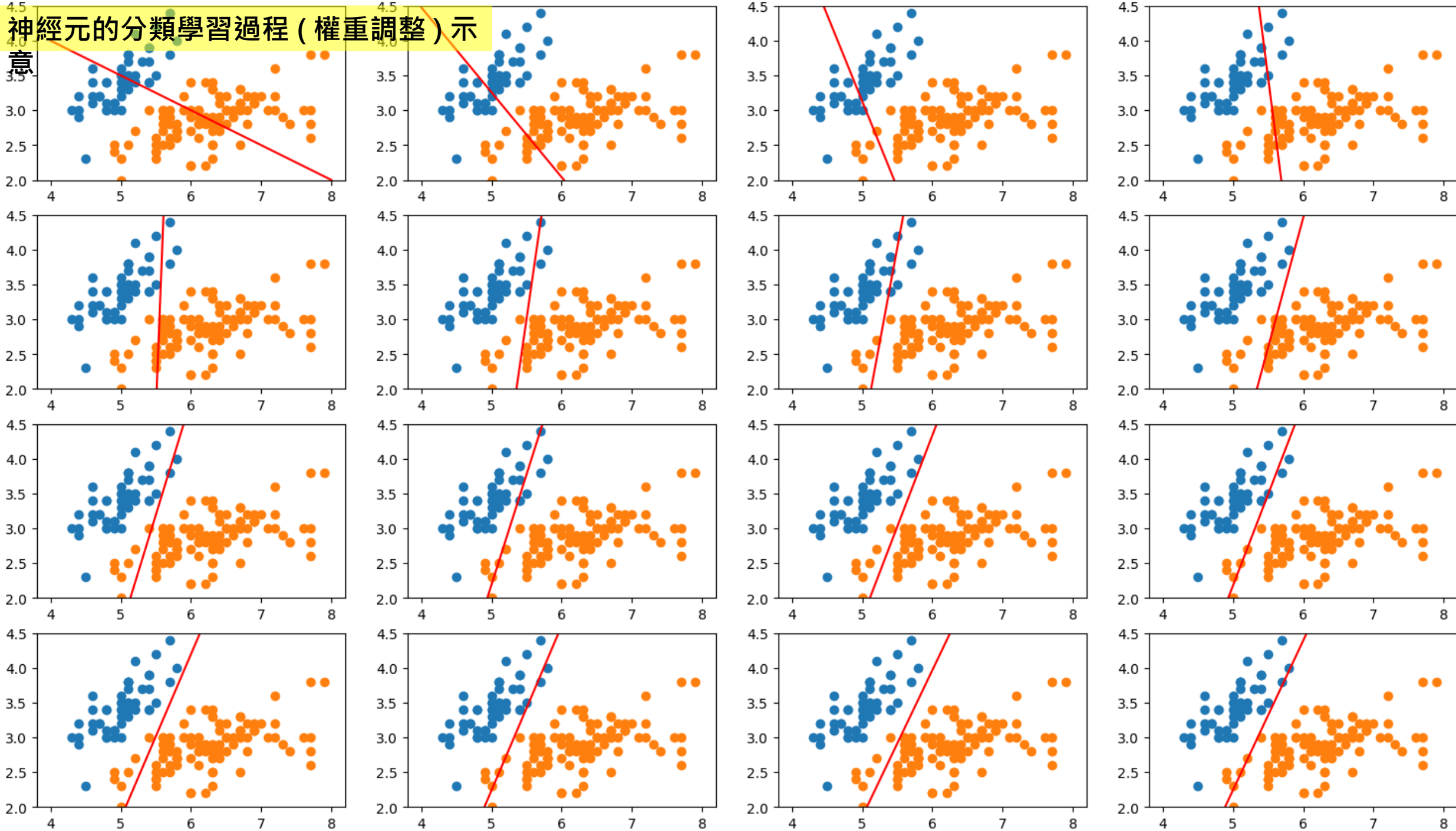
計算出該點的斜率

把權重值 w 往斜率低的方向
調整一些距離

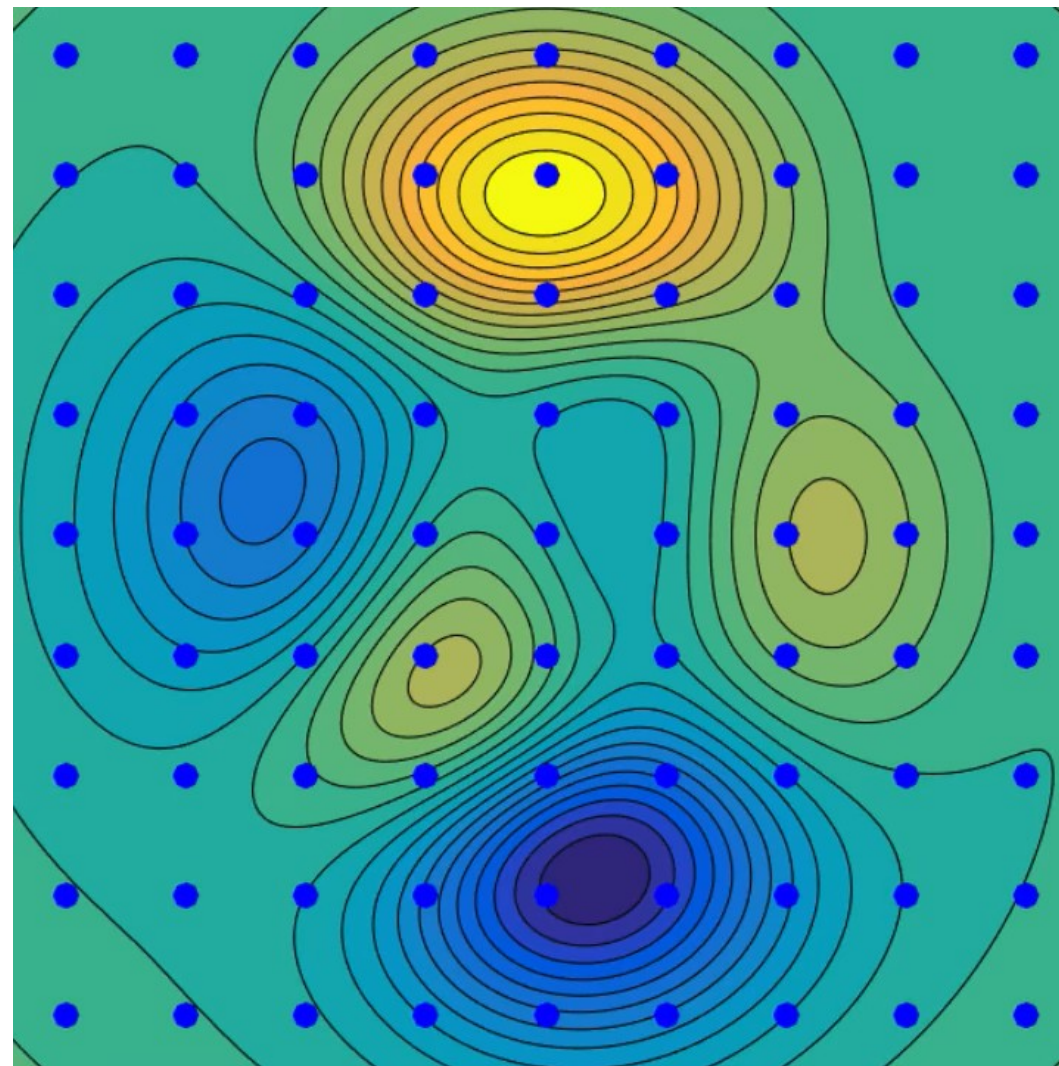
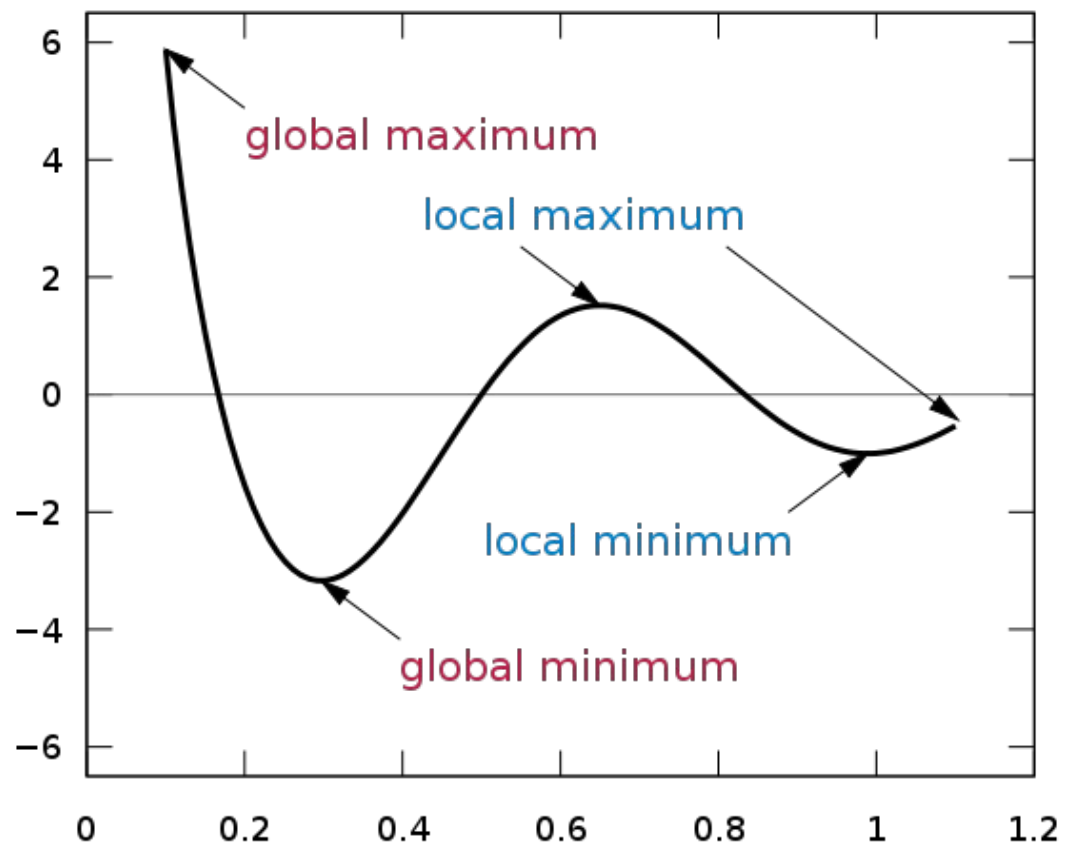
反覆的進行相同的動作，直
到誤差達到最小點為止

神經元的分類學習過程 (權重調整) 示

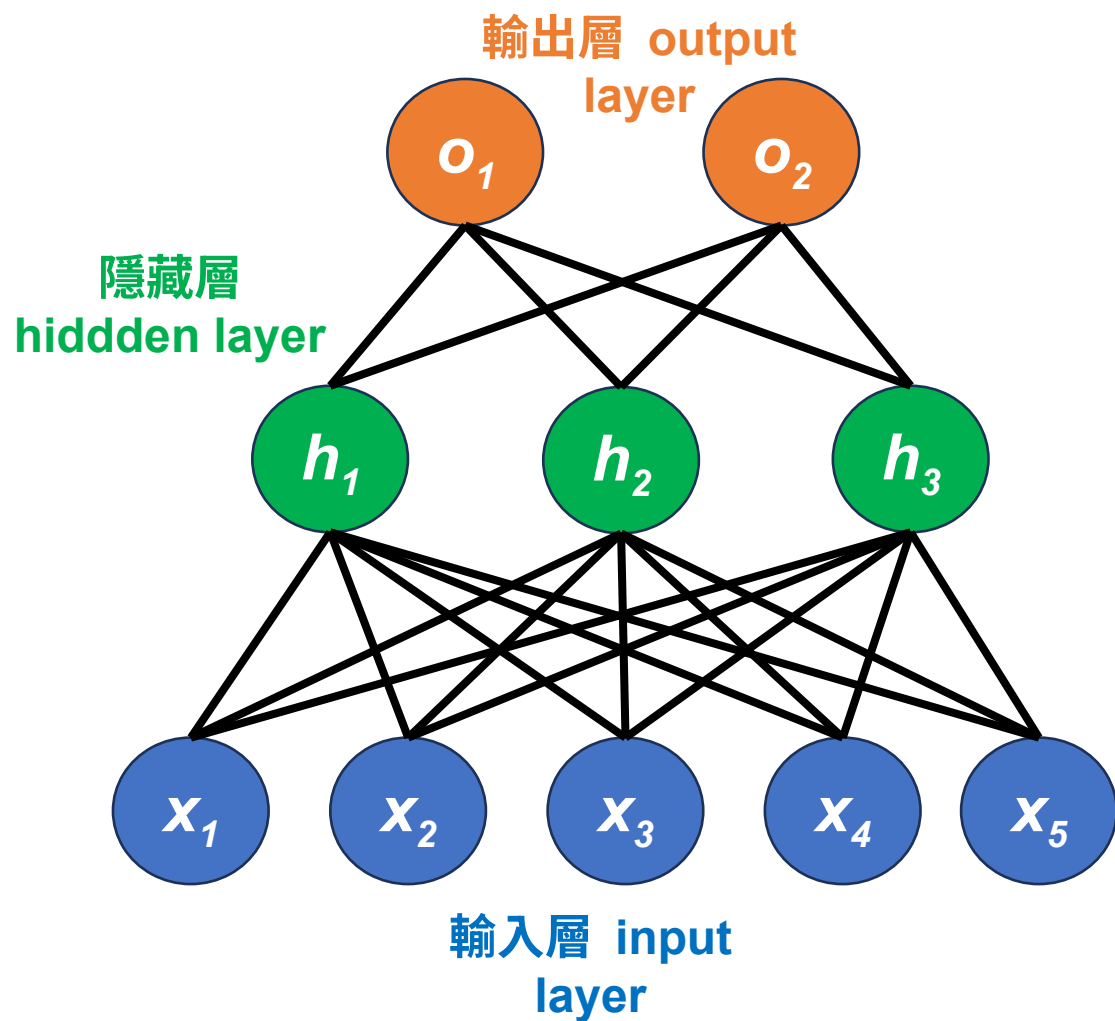
意



Local Minimum



多層類神經網路 MLP



$$o_1 = w_{41}h_1 + w_{42}h_2 + w_{43}h_3 + b_4$$

$$o_2 = w_{51}h_1 + w_{52}h_2 + w_{53}h_3 + b_5$$

$$h_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + w_{14}x_4 + w_{15}x_5 + b_1$$

$$h_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + w_{24}x_4 + w_{25}x_5 + b_2$$

$$h_3 = w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + w_{34}x_4 + w_{35}x_5 + b_3$$

用矩陣表示

$$h_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + w_{14}x_4 + w_{15}x_5 + b_1$$

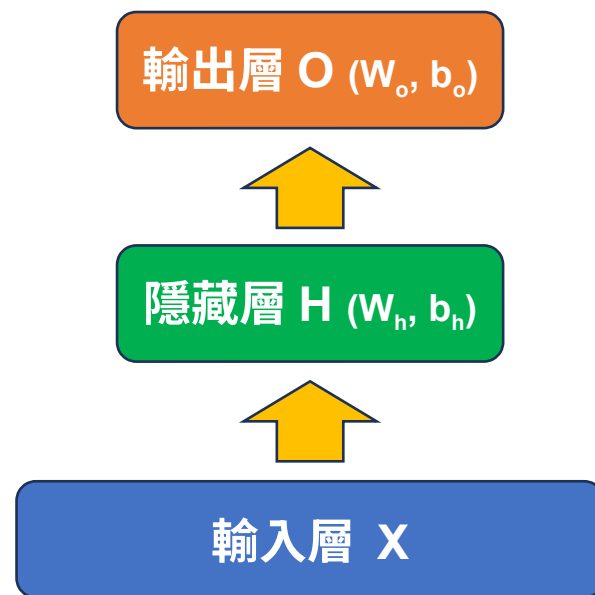
$$h_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + w_{24}x_4 + w_{25}x_5 + b_2$$

$$h_3 = w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + w_{34}x_4 + w_{35}x_5 + b_3$$

$$\mathbf{h} = \mathbf{W}_h \cdot \mathbf{x} + \mathbf{b}_h$$

$$\mathbf{W}_h = \begin{bmatrix} w_{11} & w_{12} & w_{13} & w_{14} & w_{15} \\ w_{21} & w_{22} & w_{23} & w_{24} & w_{25} \\ w_{31} & w_{32} & w_{33} & w_{34} & w_{35} \end{bmatrix} \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} \quad \mathbf{b}_h = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

因此 MLP 的一個層可以直接使用
一個矩陣 W 來表示並儲存
且矩陣運算的形式很適合使用
GPU 來進行加速



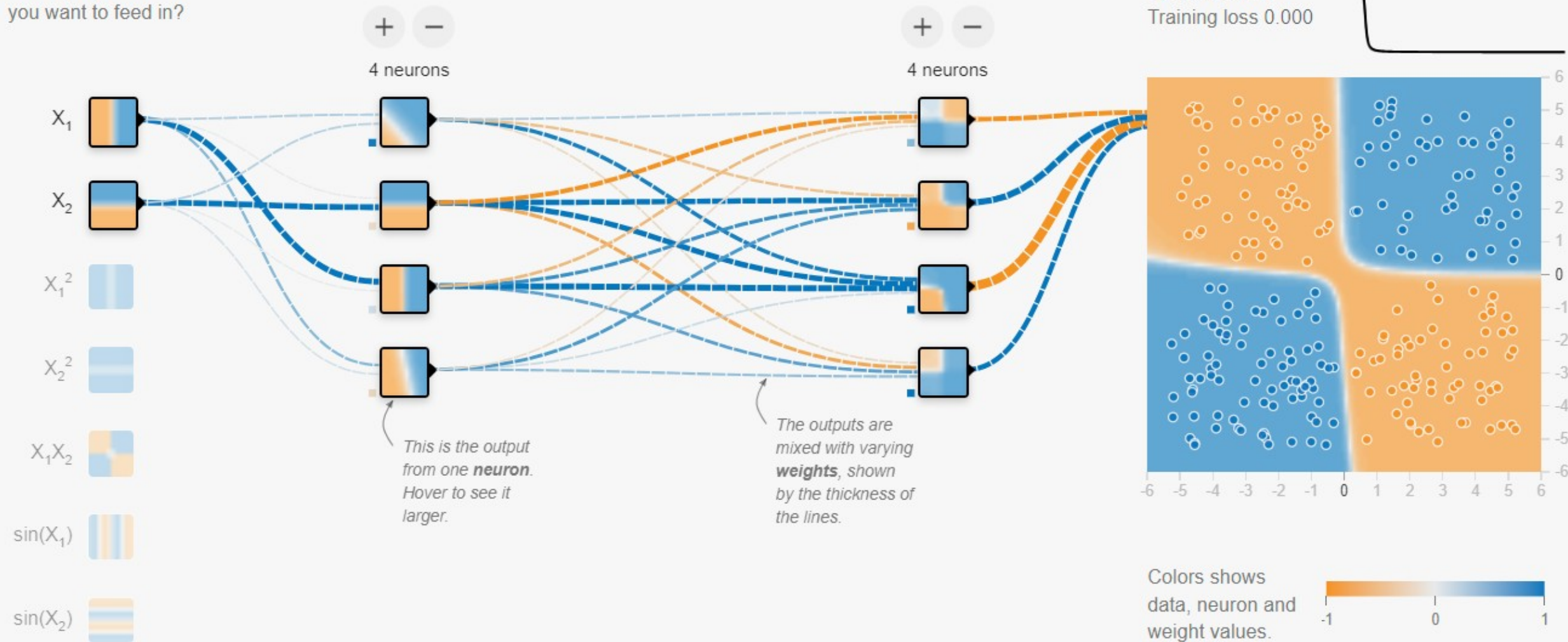
FEATURES

Which properties do you want to feed in?

+ - 2 HIDDEN LAYERS

OUTPUT

Test loss 0.000
Training loss 0.000



<https://playground.tensorflow.org/>

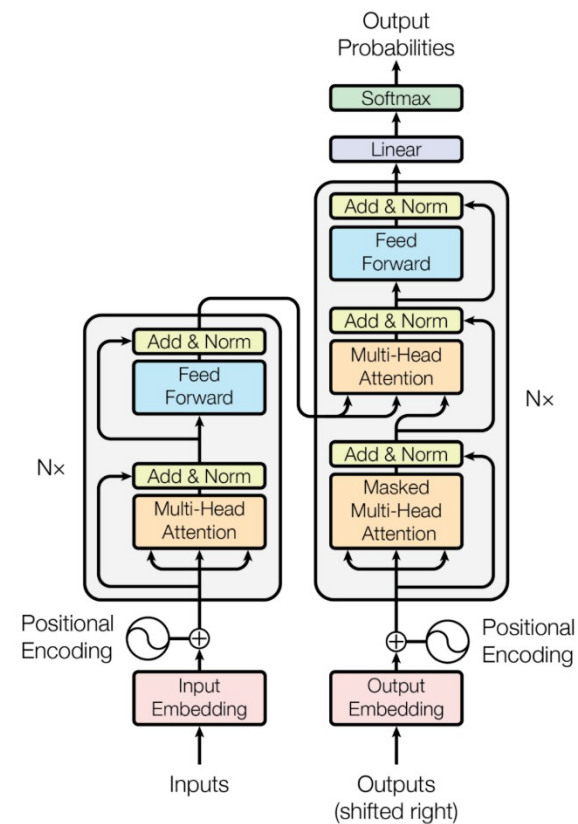
類神經網路重要事件 (1/2)

- 1943 年：第一個人工神經網路模型
 - Warren McCulloch 和 Walter Pitts
- 1958 年：感知器（Perceptron）的發明
 - Frank Rosenblatt
- 1980 年代中期：反向傳播算法（Backpropagation）的提出
 - Geoffrey Hinton、David Rumelhart 和 Ronald Williams 等人
- 1990 年代：支援向量機（SVM）等其他機器學習方法的興起
- 2006 年：深度學習（Deep Learning）復興
 - Geoffrey Hinton 等人提出了深度信念網路（Deep Belief Network，DBN）和無監督學習方法，成功地突破了神經網路的訓練難題

類神經網路重要事件 (2/2)

- 2012 年：AlexNet 的誕生
 - 由 Alex Krizhevsky 和 Geoffrey Hinton 等人開發
- 2014 年：生成對抗網路（GANs）的提出
- 2016 年：AlphaGo 的成功
- 2017 年：Google 提出了 Transformer 架構
- 2018 年：BERT 與預訓練語言模型的突破
- 2020 年：GPT-3 的推出

Transformer



LLM 的基礎架構

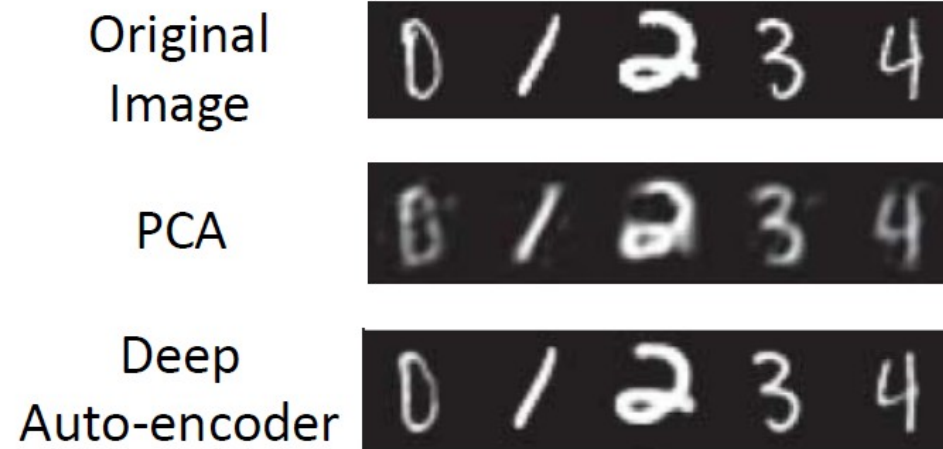
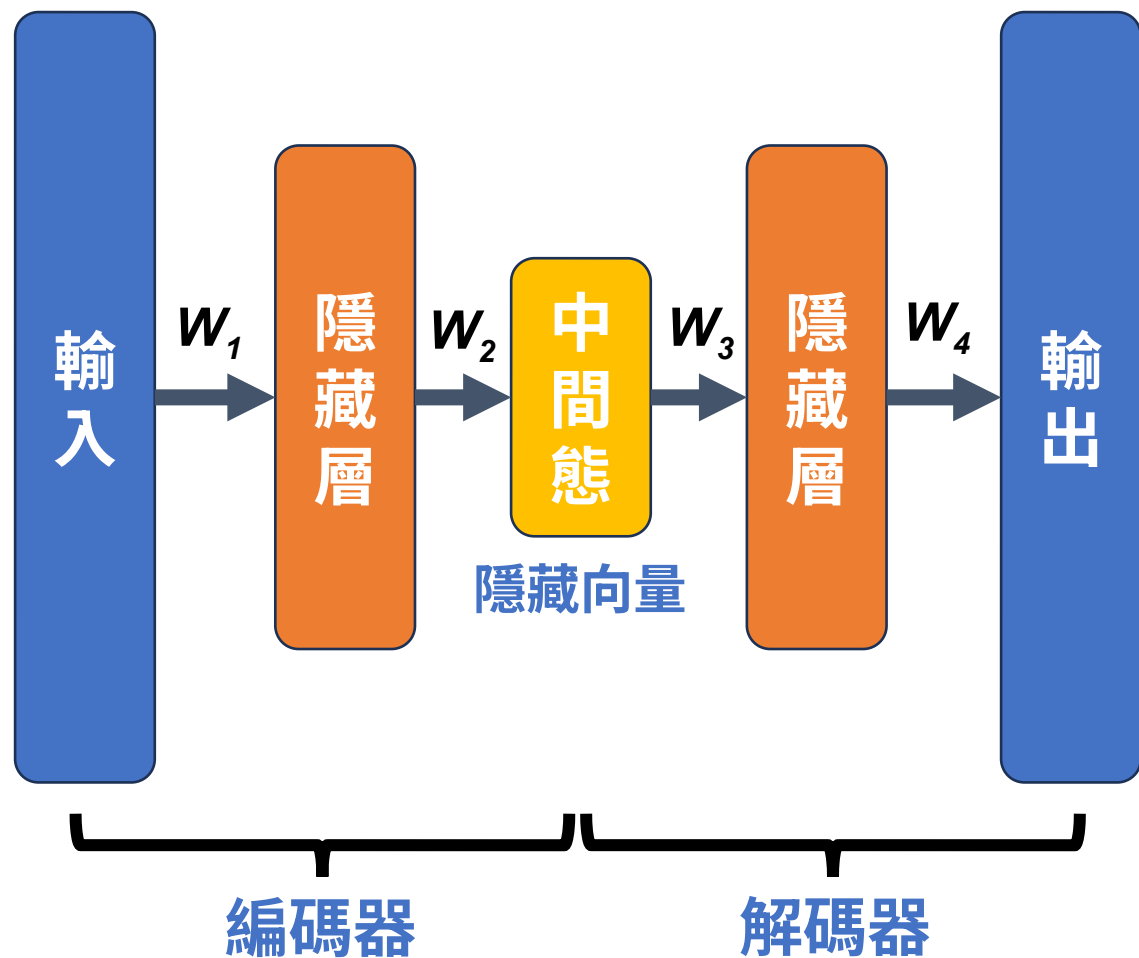
編碼與解碼架構 (The Encoder-Decoder Architecture)

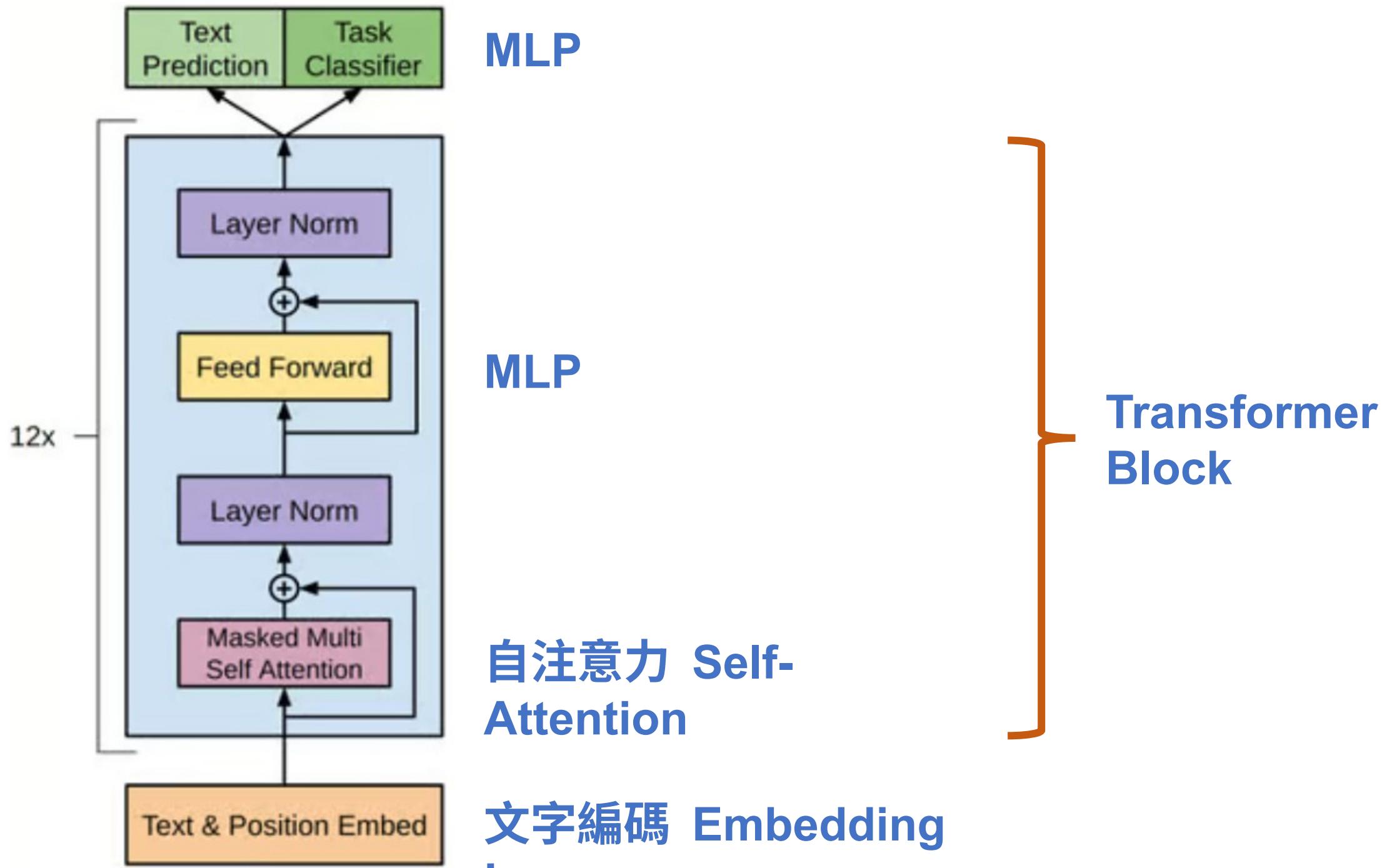


這個中間狀態可
以視為輸入的訊
息濃縮值

AutoEncoder

訓練目標：輸入等於輸出



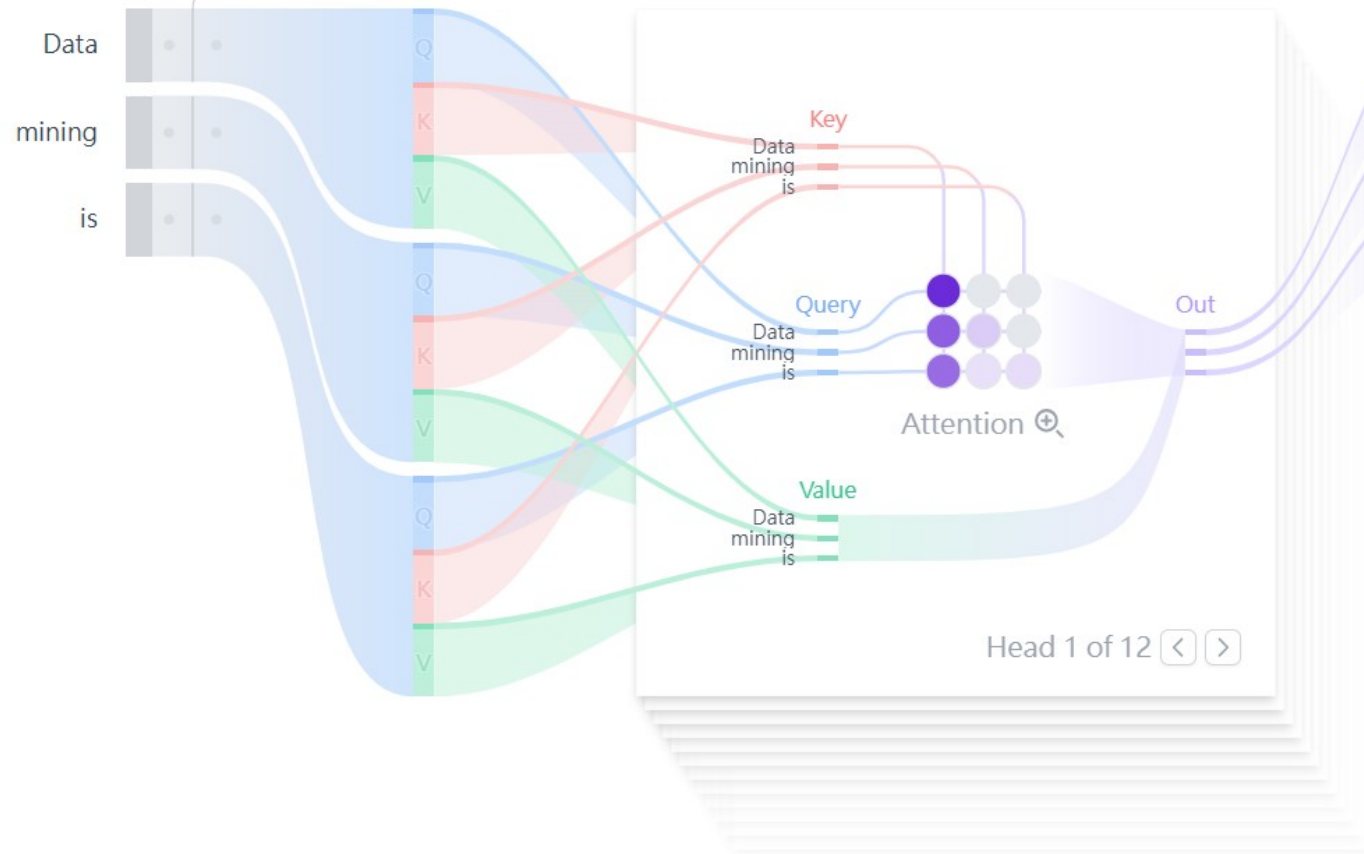


Transformer Block 1 ◀ ▶

Embedding

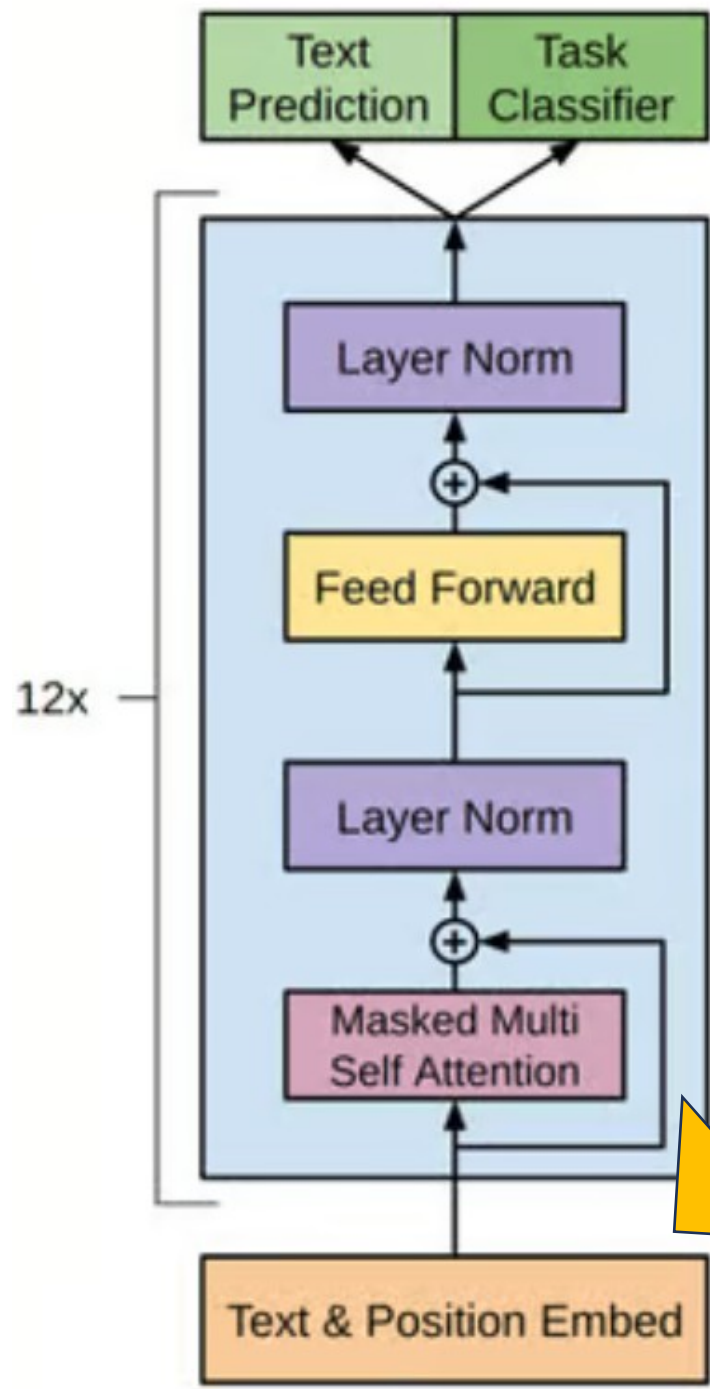
Residual

Self Attention



Probabilities 🔍

a	42.71%
not	23.76%
the	16.74%
done	8.86%
an	7.93%
still	0%
also	0%
very	0%
now	0%
currently	0%
one	0%
performed	0%
used	0%
available	0%
often	0%
more	0%



Output Probabilities

Embedding (嵌入層) :

文字輸入被分成更小的單位，稱為標記，可以是單字或子單字。這些標記被轉換成稱為嵌入的數字向量，用於捕捉單字的語義。

Attention

文字編碼 Embedding

LLM



“機器學習的”

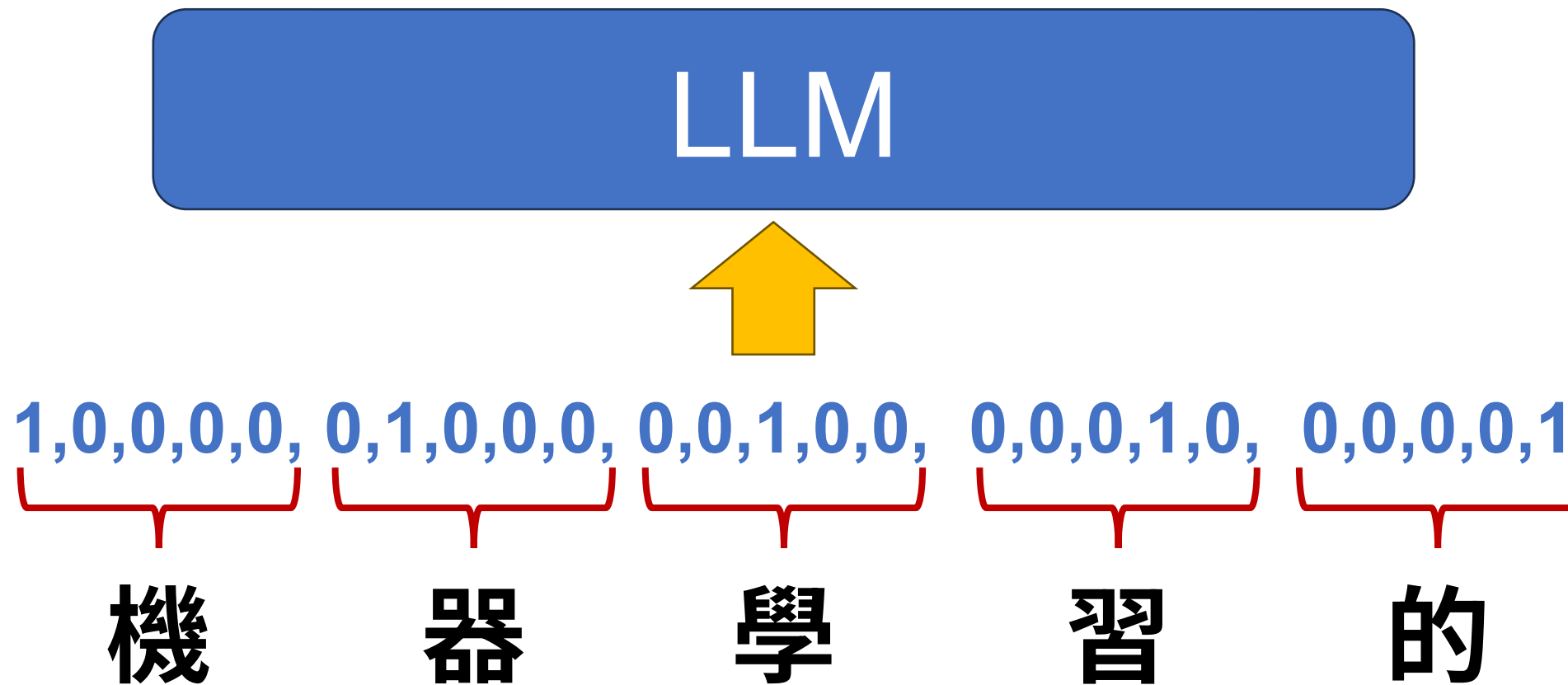


機器學習演算法只能接受數字
上面提示詞中的幾個字要如何編碼？

文字編碼

文字 (token)	編號	one hot encode
機	0	1,0,0,0,0
器	1	0,1,0,0,0
學	2	0,0,1,0,0
習	3	0,0,0,1,0
的	4	0,0,0,0,1

One Hot Encode 文字編碼



One hot encode 的缺點

- 維度爆炸
 - 每個詞都會變成一個獨立的維度，詞彙量越大，向量就越稀疏也越大。
- 無法捕捉語意關係
 - One-hot 向量之間是正交的（彼此完全無關），例如 "cat" 和 "dog" 之間的向量距離和 "cat" 與 "car" 的是一樣的。
- 無法處理未見詞
- 記憶與運算成本高

Word Vector

▼這裡只是示意，實作中欄位的意義不會這麼明確

	職級	性別	語言
字組	編碼欄位 1	編碼欄位 2	編碼欄位 3
皇帝	1	1	0
王妃	1	0	0
太監	0	1	0
宮女	0	0	0
king	1	1	1
queen	1	0	1
eunuch	0	1	1
maid	0	0	1

Embedding

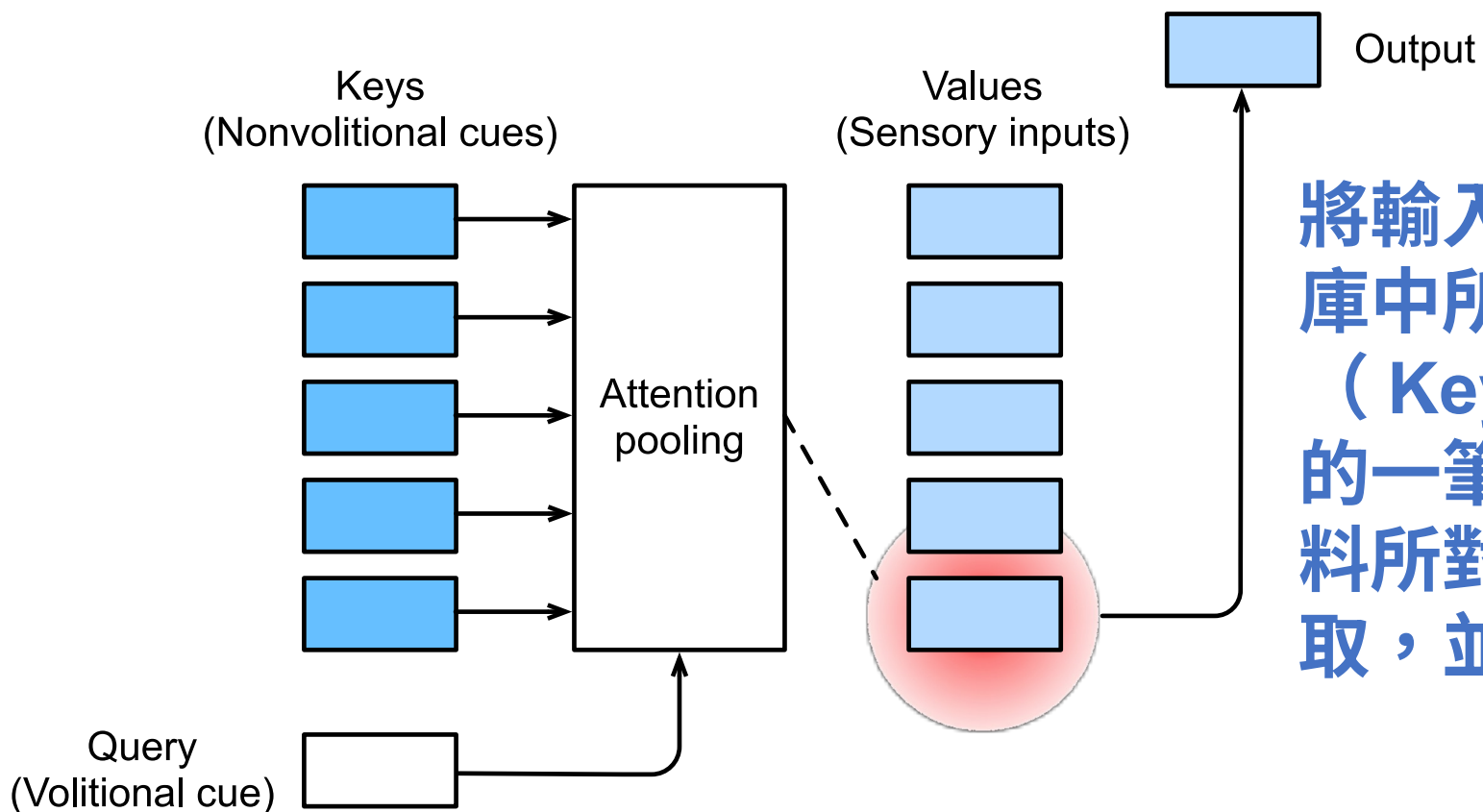
Prompt:

Data visualization empowers users to ...



GPT 的輸入資料會首先經過 **Token 編碼層**，每個 token 都會被轉換成對應的 **數值 ID**。這些 ID 在加入位置編碼資訊後，會進入 **Embedding 層**，並被映射成 **768 維的向量**表示。這些編碼後的向量，在語意上與傳統的 **詞向量 (Word Vectors)** 相似，能夠捕捉語詞間的語意與語境關係。

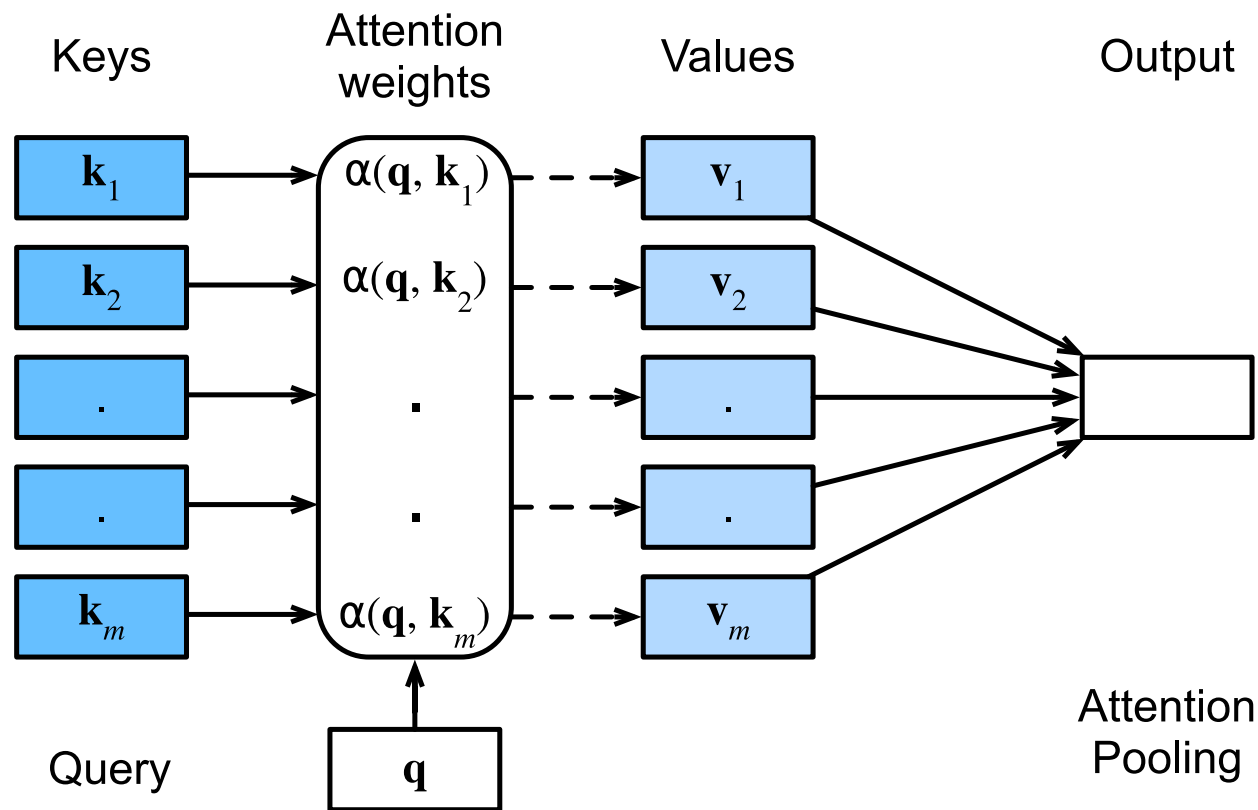
Attention



將輸入的問題（Query）與資料庫中所有範例的關鍵資訊（Key）進行比對，找出最相似的一筆資料。接著，根據該筆資料所對應的值（Value）進行提取，並輸出作為最終的結果。

KNN

Keys				Values	
花萼長度	花萼寬度	花瓣長度	花瓣寬度	類別	距離
5.6	2.7	4.2	1.3	變色鳶尾	3.12
6.3	2.5	4.9	1.5	變色鳶尾	4.02
5.4	3.9	1.7	0.4	山鳶尾	0.59
5.1	3.8	1.5	0.3	山鳶尾	0.32
6.5	3.2	5.1	2	維吉尼亞鳶尾	4.32
6.9	3.2	5.7	2.3	維吉尼亞鳶尾	5.08
Query					
5.1	3.5	1.4	0.3	?	

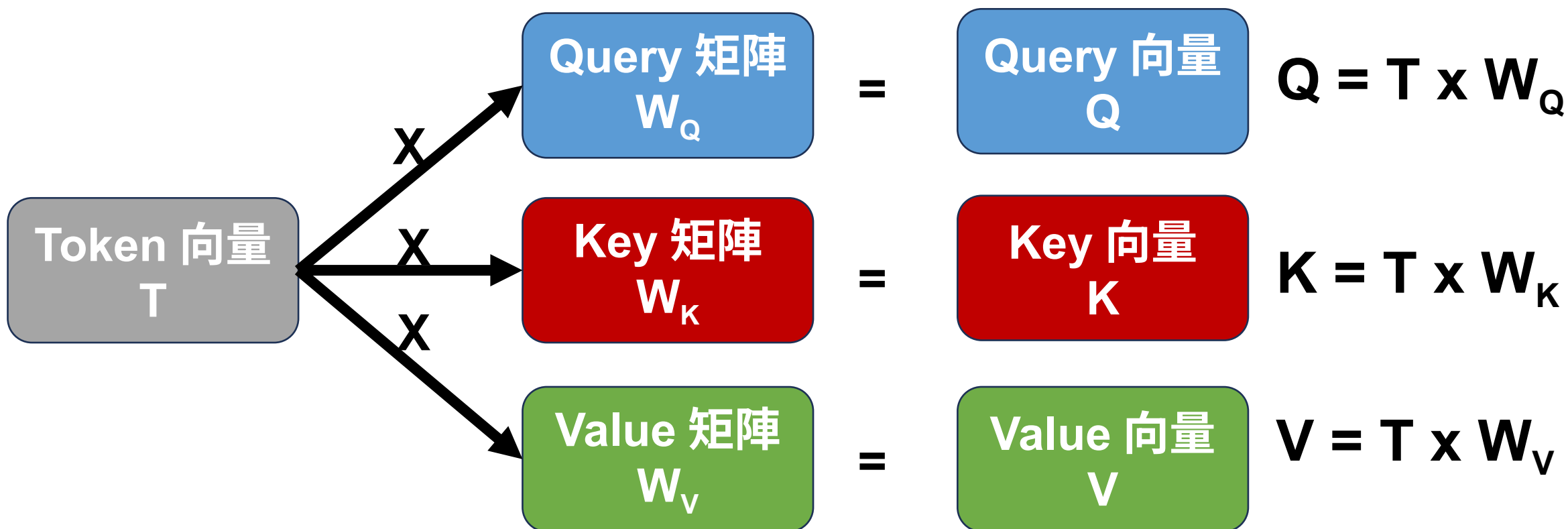


然而，在實際的運作中，**Attention** 並非僅選擇單一最相似的資料，而是計算每一筆資料的 **Key** 與 **Query** 的相似度，並將這些相似度轉換為權重。接著，將所有資料的 **Value** 根據這些權重進行加權組合，從而得到最終的輸出結果。

Self-Attention

Transformer 架構中的核心技術，用於捕捉序列中各個詞之間的關聯性

每個輸入詞（token）會同時生成 Query (Q)、Key (K) 和 Value (V) 向量



Self-Attention 的運作方式是：

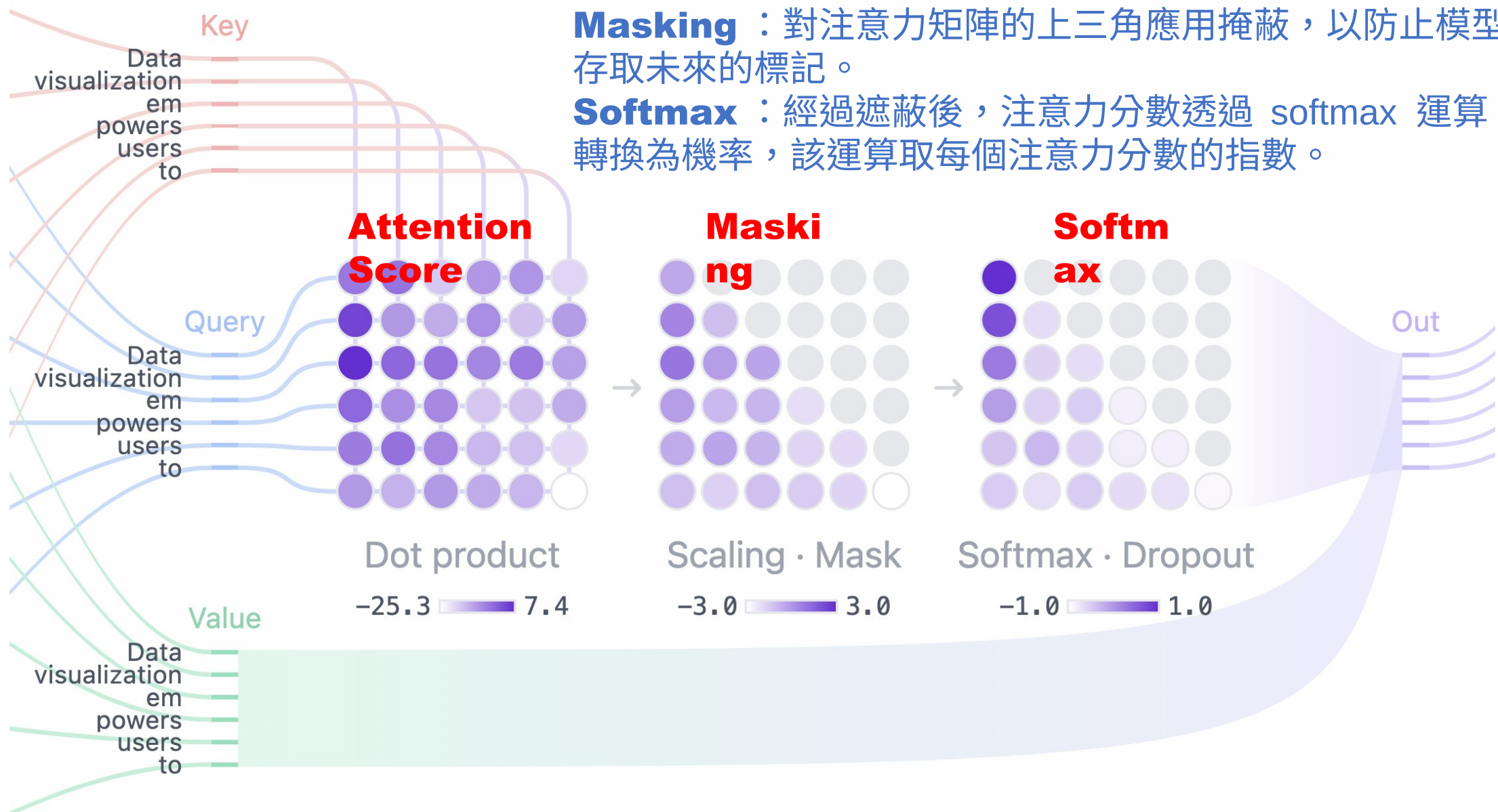
對於序列中的每個詞，將其作為查詢（Query），並與序列中所有詞的鍵（Key）進行相似度比對，以判斷該詞在理解當前詞語時的重要程度。接著，根據這些相似度分數（權重），對所有詞的值（Value）進行加權總和，從而生成該詞的新表示向量。

在實作上，這個過程通常包括以下步驟：

- 將輸入向量映射為 Query、Key、Value。
- 計算 Query 與所有 Key 的點積相似度。
- 對相似度進行 softmax 正規化，得到權重分佈。
- 將所有 Value 向量依照權重加權平均，得到輸出。

可讓模型理解詞與詞之間的關係，不受距離影響（例如一句話中的長距依賴關係）

Masked Self-Attention



Attention Score：Query 和 Key 矩陣的點積決定了每個 Query 與每個 Key 的對齊方式，從而產生一個反映所有輸入標記之間關係的方陣。

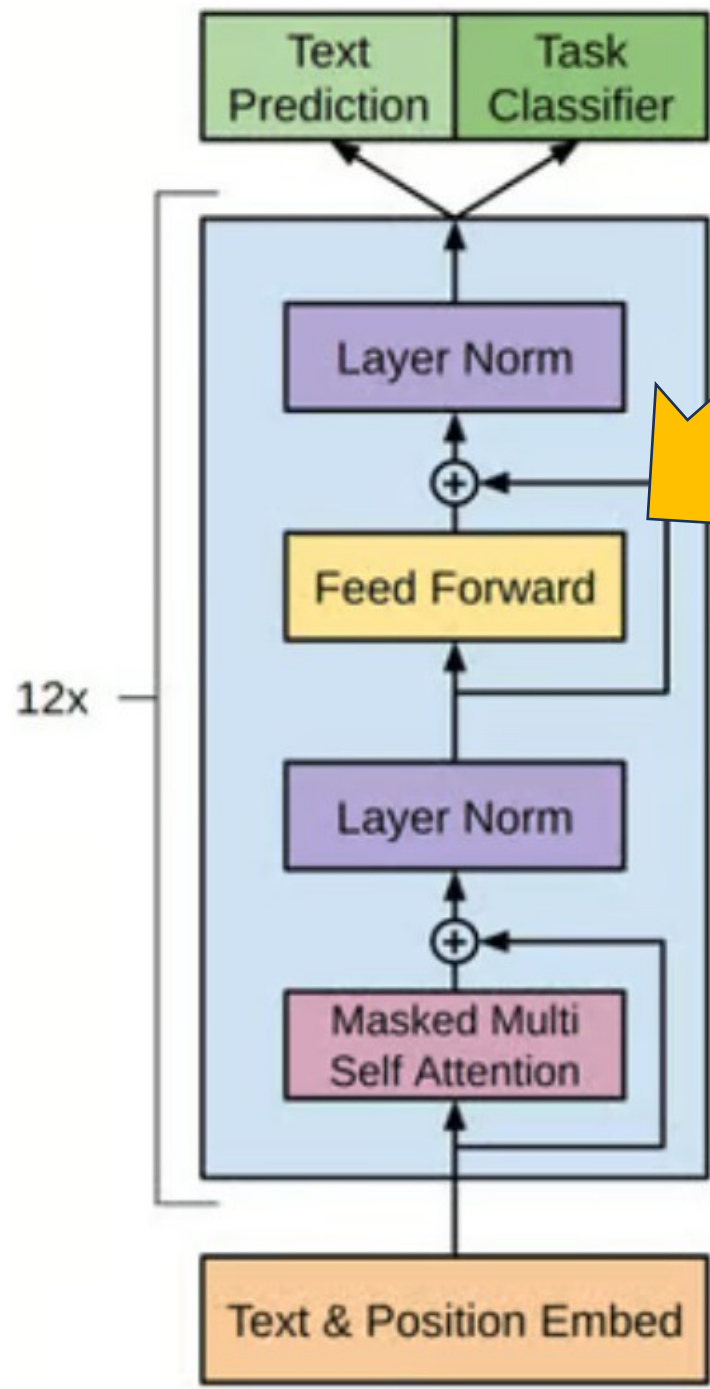
Masking：對注意力矩陣的上三角應用掩蔽，以防止模型存取未來的標記。

Softmax：經過遮蔽後，注意力分數透過 softmax 運算轉換為機率，該運算取每個注意力分數的指數。

提示詞：「小明看到小華後向他揮手。」



1. 傳統模型（如 RNN）可能會困惑「他」指的是誰（小明還是小華）
2. Transformer 透過 Self-Attention 機制，會：
 - 計算「他」與「小明」的關聯程度。
 - 同時也計算「他」與「小華」的關聯程度。
3. 若語境中是「小明看到小華後向他揮手」，語意更傾向「他」指的是「小華」。
4. Attention 就會給「小華」更高的權重，幫助模型正確理解指代關係。



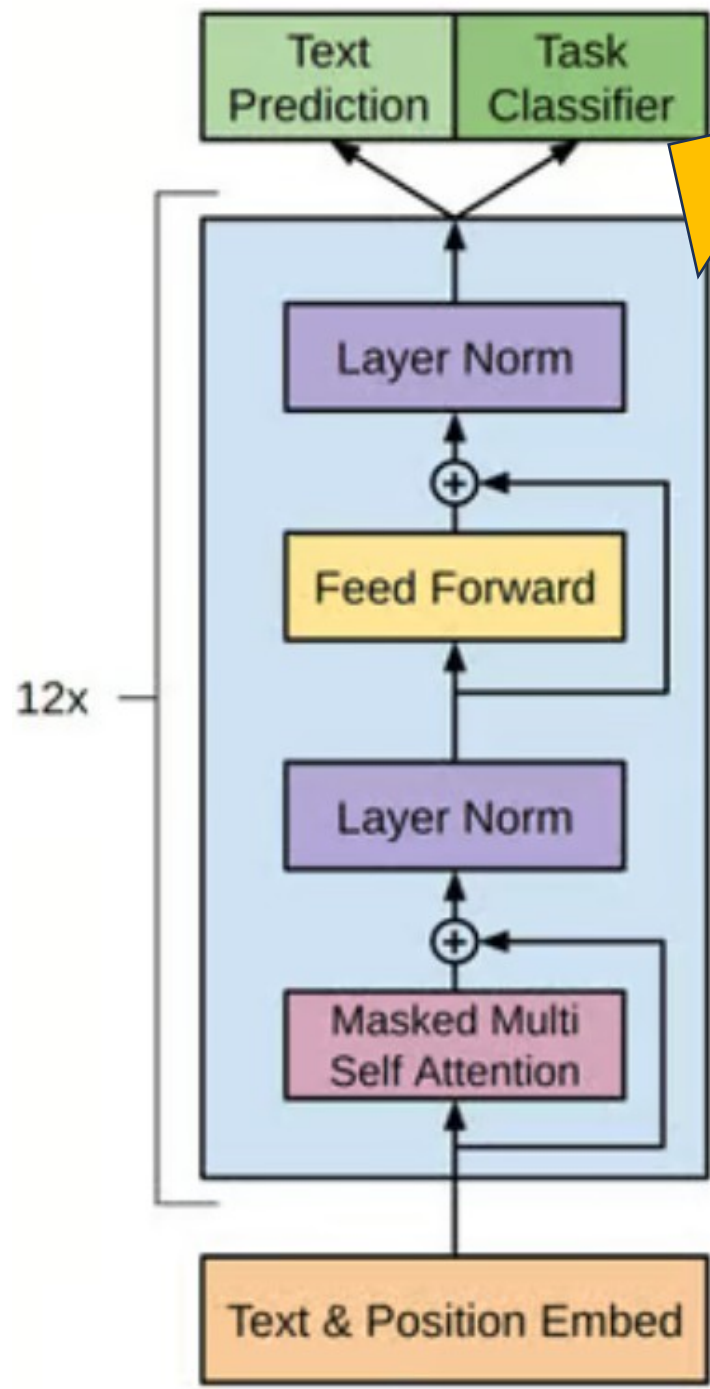
Output Probabilities

MLP (Multilayer Perceptron) Layer :
MLP (多層感知器) 層，一個對每個標記獨立操作的前饋網路。雖然注意層的目標是在標記之間路由訊息，但 MLP 的目標是細化每個標記的表示。

Block

自注意力 Self-Attention

文字編碼 Embedding



Output Probabilities

Output Probabilities (輸出機率) :
最後的線性層和 softmax 層將處理後的
嵌入轉換為機率，使模型能夠對序列中的
下一個標記做出預測。

MI

Transformer
Block

自注意力 Self-
Attention

文字編碼 Embedding

Output Probabilities 輸出機率

模型會根據輸出的結果，為詞彙表中的每個標記分配一個機率。這些機率表示每個標記成為序列中「下一個字」的可能性大小。

Temperature ? Sampling ? ☒ Top-k ☐ Top-p

0.8 k5

Tokens	Logits	Scaled logits	Top-k	Softmax
visualize	-135.91	-169.89	-169.89	54.67%
create	-136.68	-170.85	-170.85	20.87%
see	-137.12	-171.40	-171.40	12.09%
make	-137.65	-172.06	-172.06	6.26%
easily	-137.67	-172.08	-172.08	6.11%
quickly	-137.72	-172.15	$-\infty$	0%
explore	-137.78	-172.23	$-\infty$	0%
find	-138.05	-172.56	$-\infty$	0%

KV Cache

PROMPT : “**Data mining is a technique that**”

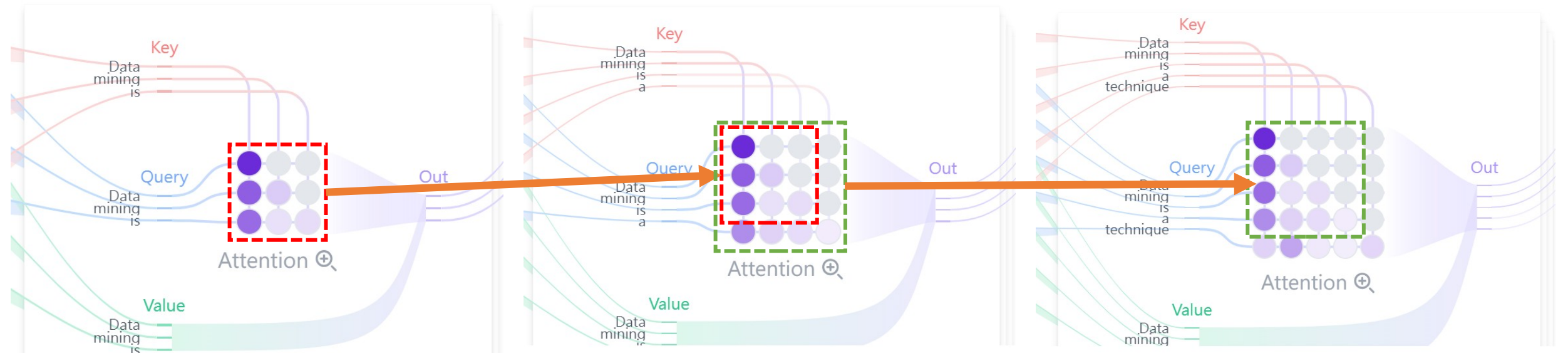
Turn 1



Turn 2



Turn 3



KV Cache

- 優點
 - 將產生每個 token 所需的時間複雜度降低到線性
- 缺點
 - 隨著序列的長度線性增加，所需的**記憶體也會越來越大**
 - 每個單獨的對話都需要保存自己的 KV Cache，而**不同的對話很難重用它們**

LLM 模型訓練

大型語言模型是如何從零開始訓練

LLM 的訓練流程



預訓練 Pretraining

目標：學會語言的基本知識與邏輯

使用大量公開網路文字（如 Wikipedia、書籍、網站）

模型任務：根據前文預測下一個字

例：「今天天氣真 ____」 → 模型學會填「好」、「糟」等合理詞

GPT-3

訓練語料總量：約 570 GB 的純文字

大約包含：3000~4000 億個「字詞單位」
(tokens)

微調 Finetuning

目標：讓模型更擅長特定任務

用較小但有標註的資料集（例如問答對、對話紀錄）

例如：法律、醫療、客服等領域語言

模型學會：「如何用對的格式回答問題」、「用更有用的方式寫作」

GPT-3 微調使用

約 13,000 條高品質任務式指令資料

資料來源：人工標註對話，包含問答、摘要、翻譯、推理等任務

對齊訓練 Alignment

又叫「強化學習人類反饋」

(RLHF)

對話式模型的關鍵

人類會評價模型回答好不好

模型會學習如何回應「更合乎人類喜好」的回答方式

像是老師糾正學生：「這樣講比較有禮貌」、「這樣比較精確」

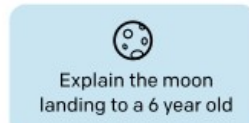
InstructGPT

Reinforcement Learning from Human Feedback

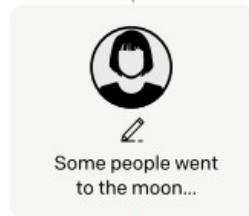
Step 1

**Collect demonstration data,
and train a supervised policy.**

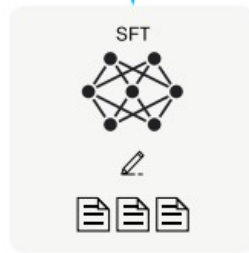
A prompt is
sampled from our
prompt dataset.



A labeler
demonstrates the
desired output
behavior.



This data is used
to fine-tune GPT-3
with supervised
learning.

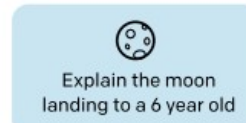


dataset : 13K

Step 2

**Collect comparison data,
and train a reward model.**

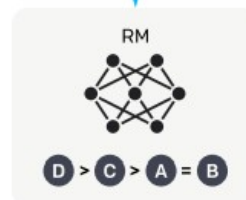
A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.



This data is used
to train our
reward model.



dataset : 33K

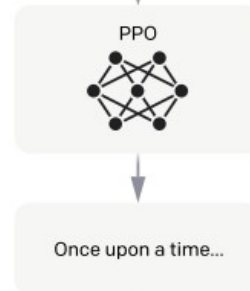
Step 3

**Optimize a policy against
the reward model using
reinforcement learning.**

A new prompt
is sampled from
the dataset.



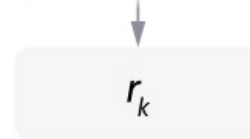
The policy
generates
an output.



The reward model
calculates a
reward for the output.

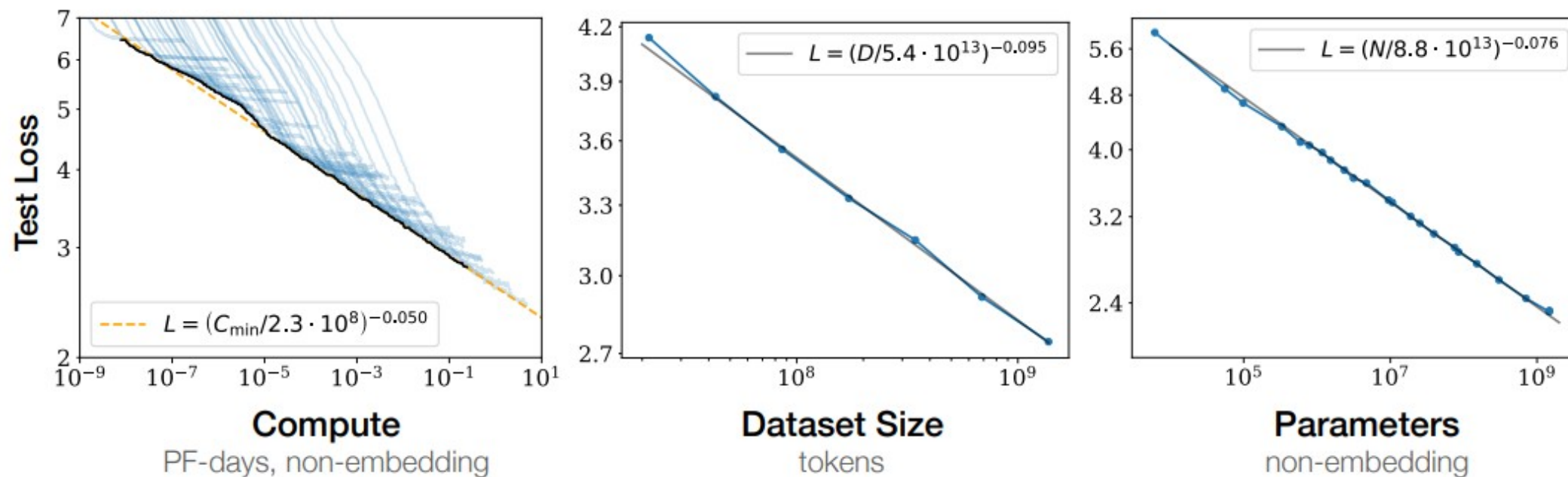


The reward is
used to update
the policy
using PPO.



dataset : 31K

LLM Scaling Laws



當資料、參數與計算量按對數規模成長，模型損失呈現 近似冪律
(power-law) 下降

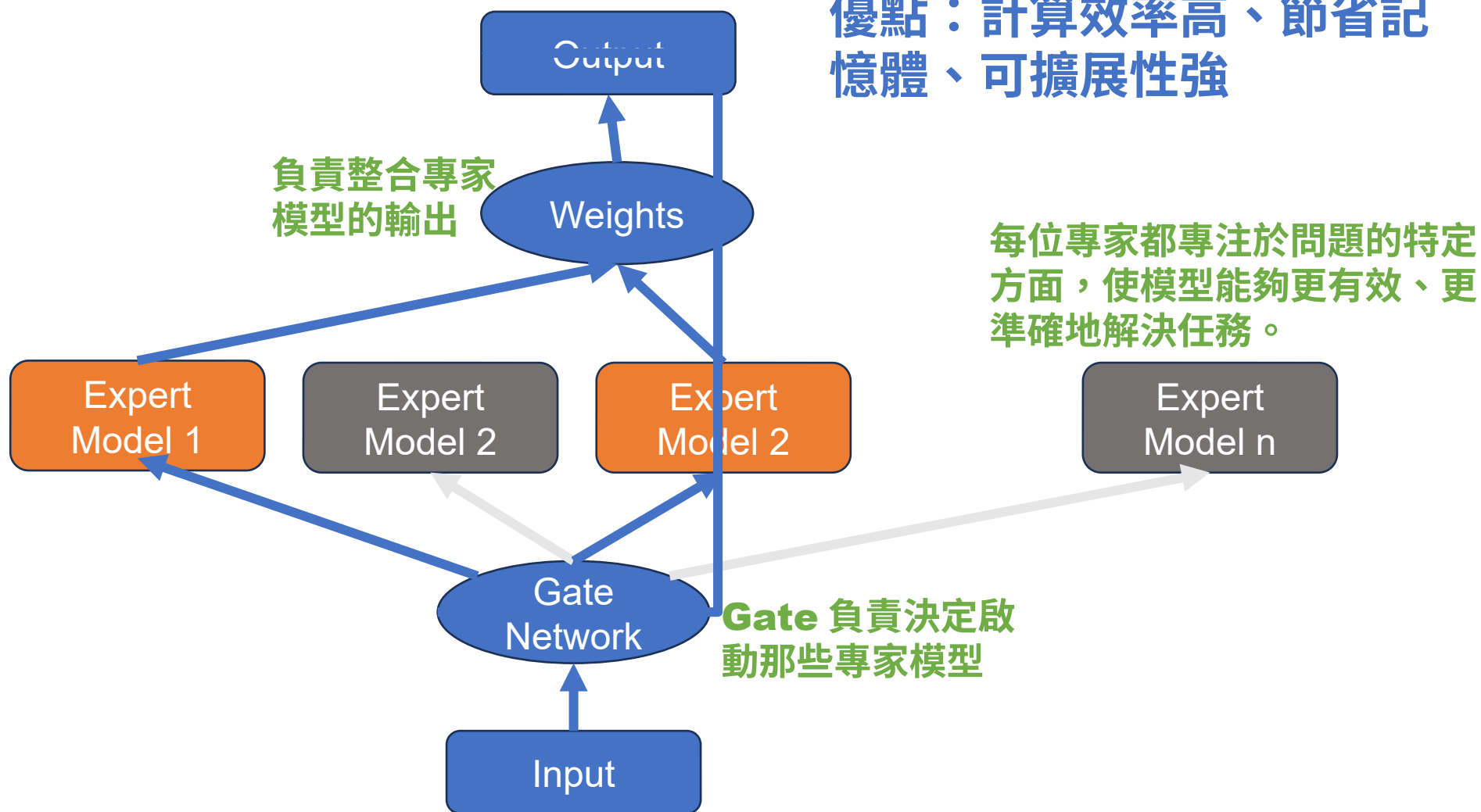
影響：運算成本高、訓練與推論速度慢

解決方案：讓部分模型參與運算 ☒ MoE (專家混合模型)

MOE Architecture

專家混合 (MoE) 技術將大型模型分解為較小的專用網路

優點：計算效率高、節省記憶體、可擴展性強





LLM 實作

LLM 基本操作

深度學習模型推理引擎

- Pytorch, Tensorflow
 - Hugging Face Transform
- llama.cpp
 - GGUF
- ONNX
- TensorRT, TensorRT-LLM

LLM 推理伺服器

- vLLM
 - 高效 GPU 批次推理
- Ollama, LM-Studio
 - 個人或小企業輕量級
- Triton Inference Server
 - 企業級 GPU 分布式推理

GGUF

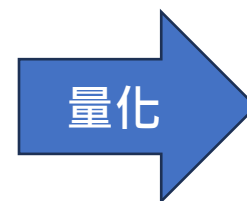
- GPT-Generated Unified Format
- 用來儲存和部署大型語言模型的一種高效格式
- 主要應用在像是 llama.cpp 等輕量本地運行的 AI 推理工具中
- 主要目的
 - 讓語言模型更容易本地運行
 - 降低資源消耗（RAM/VRAM）
 - 提高模型讀取速度與通用性

GGUF 檔名意義

- **Qn**：量化用的位元數
 - **K-Quants** 變體
 - **K**：通常表示這是一個 K-Quants 方案。
 - **S** (Small)：指示該 K-Quants 變體使用了較小的區塊大小或其他優化，可能在某些硬體上提供更好的效能。
 - **M** (Medium)：指示該 K-Quants 變體使用了中等的區塊大小或其他優化。
 - **O**：優化過的量化版本，通常針對特定硬體或推理需求進行優化。
- 範例：**gemma-2-2b-it-Q4_K_M.gguf**

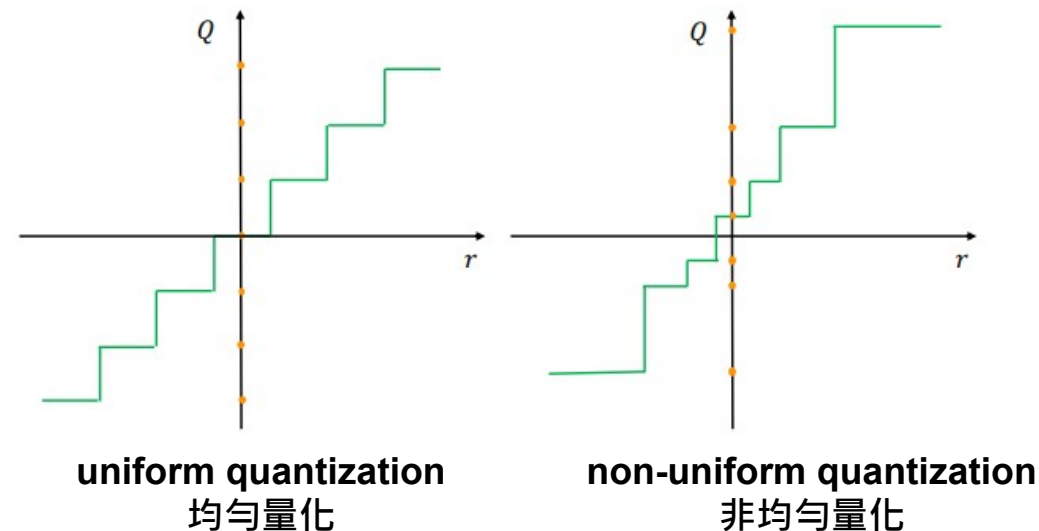
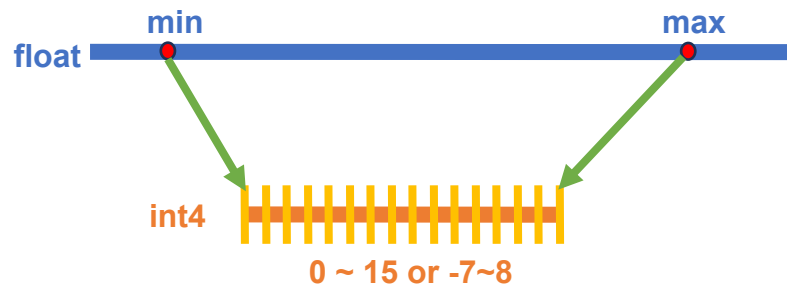
模型量化 Quantization

- 將大型語言模型中的浮點數權重 (fp32, fp16) 轉換成較低位元 (8, 4 bit) 數值格式的過程
- 目標
 - 減少模型大小
 - 提升推理速度
 - 節省記憶體與能源



Quantization 方法

- 浮點轉整數：3.1415926 $\rightarrow$ 3
 - 會有溢位問題
- Scale 法



由於神經網路中的權重多數集中分布於中間區段，因此可採用非線性且非均勻的量化策略，以更有效地保留重要資訊。

NF4 轉換範例

這裡對應表的數值是非線性分布

NF4 = [-1.0000, -0.6962, -0.5251, -0.3949, -0.2844, -0.1848, -0.0911, 0.0000, 0.0796, 0.1609, 0.2461, 0.3379, 0.4407, 0.5626, 0.7230, 1.0000]

Quantization

Input: [0.32, -1.76, 0.025, -1.22]

Divide by absmax (1.76)

[0.1818, -1.0000, 0.0142, -0.6932]

Round to the nearest based on NF4

[0.1609, -1.0000, 0.0796, -0.6962]

Store the corresponding index

[9, 0, 8, 1]

Dequantization

[9, 0, 8, 1]

Find the corresponding value

[0.1609, -1.0000, 0.0796, -0.6962]

Multiply with constant absmax

0.32, -1.76, 0.025, -1.22]

ollama

- Ollama 是一個讓使用者可以**本地運行大型語言模型（LLMs）**的平台
- 下載 <https://ollama.com/download>
- 執行命列 `ollama run llama3.2`
- 更多模型 <https://ollama.com/search>
- Page Assist：支持 ollama 的 chrome 插件
 - <https://chromewebstore.google.com/detail/page-assist-a-web-ui-for/jfgfii-gpkhlkbnfnbobbkinehhfdhndo>

ollama 基本指令

- llama run < 模型名稱 >
 - 啟動指定模型並開始互動，例如 `ollama run llama3.2`
- ollama pull < 模型名稱 >
 - 從 Ollama 官方模型庫下載模型，例如 `ollama pull mistral`
- ollama list
 - 列出目前本地端已安裝的模型
- ollama remove < 模型名稱 >
 - 移除指定的本地模型，例如 `ollama remove llama3.2`
- ollama show < 模型名稱 >
 - 顯示模型詳細資訊（包含配置、版本、大小等）
- ollama create < 自定名稱 > -f Modelfile
 - 依照 Modelfile 建立自定義模型

LLM 提示詞工程

Prompt Engineering

Prompt Engineering

- 提示詞工程是指設計和優化自然語言提示 (prompts) ，來引導 LLM 產生期望的、高質量和相關的輸出。
- 透過有策略性地撰寫、設計或結構化提示詞 (prompts) ，來引導大型語言模型（如 GPT-4）生成符合預期的輸出內容的過程。
- 關鍵要素
 - 提示詞的格式與語言風格
 - 上下文與範例的提供 (Few-shot learning)
 - 限制條件與目標明確化

提示詞的格式與語言風格

- 什麼是提示詞？
 - 提示詞是給 AI 的指令或問題
 - 用來引導模型產生所需的回答、圖像或程式碼等
- 常見結構：
 - 角色設定（ Role ）：你是一位 ...
 - 任務描述（ Task ）：請幫我完成 ...
 - 輸出格式（ Format ）：
 - 請用清單 / 表格 / Markdown...
 - 語氣要求（ Tone ）：請用專業 / 親切 / 幽默的語氣 ...

範例：你是一位專業日文翻譯，請將以下句子翻譯成自然且禮貌的日文。

Completion vs Chat Mode

- 使用提示詞工程，請一定要使用具備 chat template 的 **Chat Mode**
- 差異
 - Completion mode
 - LLM 模型不會執行提示詞中的指令，而是一直在進行文字接龍
 - Chat mode
 - LLM 模型可以良好的執行提示詞中的指令

Gemini Gem

- Gem 是 Google I/O 曾預覽過的功能，是可客製化版本的 Gemini。用戶可針對特定主題或目的客製化成特殊主題專家。
- <https://gemini.google.com/gem>

測試用 Prompt：

你是一個人資輔助系統，可以協助人資專員評估面試者是否適合公司的工作。第一步先要求使用者提供工作需求。當使用者提供工作需求後，才再要求提供履歷。然後根據履歷跟工作需求來分析這位面試者是否跟這份工作匹配，輸出的格式要包含：是否合適：是或否
合適度：0~10 分分析：分析結果當分析完畢後，可以要求使用者再提供下一個面試者的履歷

RAG

檢索增強生成
Retrieval Augmented Generation

RAG 基本概念

當我們問 LLM 模型的問題不在訓練資料集中時。

You

陳仁政的履歷

比較早期的模型會有幻覺，產生不存在的訊息。

Assistant

meta-llama-3.1-8b-instruct

陳仁政 (Randy Chen) 是一位出色的臺灣資訊安全專家。他擔任過多項重要職務，包括：

1. 臺北市政府數位發展局副局長：2020年至2023年間，陳仁政負責台北市政府的數位發展事務。
2. 全國性公共機關數位安全中心執行長：2019年至2020年間，陳仁政擔任全國性公共機關數位安全中心（NPCISC）的執行長，這是臺灣一家負責公共機關數位安全相關事務的中心。
3. 臺北市政府資訊局副局長：2017年至2019年間，陳仁政在台北市政府資訊局擔任副局長。

除了公職以外，陳仁政也活躍於企業界。他曾擔任過多家公司資安顧問或董事。

You

陳仁政的履歷

Assistant

llama-3.2-3b-instruct

比較新的模型可以直接回答不知道



Sorry，我們不知道有關陳仁政的資訊。能夠提供更多的背景或詳情嗎？

< 1/3 > ↺ 📄 ✎ 🗑

You

陳仁政的履歷

最近新的模型可以提供建議



Assistant `gemma-3-1b-it`

由於我無法存取網際網路資料，因此我無法提供您陳仁政的完整履歷。

不過，我可以根據一般資訊，協助您找到他履歷：

1. **Google 搜尋:** 您可以嘗試在 Google 上搜尋「陳仁政 電資履歷」或「陳仁政 LinkedIn」來取得相關資訊。
2. **人力銀行:** 您可以查詢人力銀行 (人力銀行網站) 上陳仁政的資料，但可能需要使用專業工具才能找到詳細的履歷。
3. **相關新聞/媒體報導:** 搜尋是否有關於陳仁政的新聞報道或媒體報導，這通常會提供他目前的職務和經驗。

如果您想讓我幫您查找特定資訊（例如：最近的職位、公司等），請告訴我您需要什麼樣的資訊，我可以試著協助您查詢。

ChatGPT 則是提供連網功能

根據公開資訊，名為「陳仁政」的人士有多位，以下是其中幾位的簡要履歷：

1. **人工智慧與資料分析專家**：陳仁政博士畢業於中原大學電子研究所，曾在中央研究院資訊科學研究所從事博士後研究，並在吉鴻電子擔任資深工程師。他在人工智慧、機器學習、電腦視覺、影像處理等領域有豐富經驗，現任台灣人工智能產業協會講師、實踐大學推廣中心講師及緯育TibaMe講師。

Google 雲端硬碟 +2

2. **製造工程經理**：在LinkedIn上，有一位名為陳仁政的人士擔任IChia Technologies, Inc的製造工程經理，負責汽車電容式觸控研發和售前支持，以及監督部門人員工作、規劃生產線、改善製程等。

LinkedIn

3. **資安顧問**：另一位陳仁政在創逸科技服務有限公司擔任資安顧問，擁有超過30年的資訊管理經驗，專注於資訊系統、網路、機房的建置、維運、資訊安全管理與稽核等。

Cake +1

4. **教育界人士**：根據維基百科，陳仁政曾於2002年受臺南縣政府委派，籌備並擔任臺南縣立沙崙國民中學（現為臺南市立沙崙國際高級中等學校）的首任校長，任期為2005年8月至2007年7月。

維基百科，自由的百科全书



訊息 + 問題 = 提示詞

如果我們把訊息跟問題一起當成 prompt 丟給 LLM 模型時：

CV = 陳仁政的個人簡歷

學術與專業背景：
博士學位 | 中原大學電子研究所
博士後研究 | 中央研究院資訊科學研究所

產業與教育經歷：
資深工程師 | 吉鴻電子
正工程師 | 冠捷科技
資料科學家 | 104 人力銀行人資學院
兼任實務教師 | 長庚大學工商管理系
顧問 | 104 人力銀行人資學院

現職：
講師 | 台灣人工智能產業協會
講師 | 實踐大學推廣中心
講師 | 緯育 TibaMe

專案經驗：
智慧金融 | 收鈔機韌體開發 (偽鈔偵測)
消費電子 | 電視韌體開發
智慧交通與監控
高承載管制違規偵測系統
手持式雷射測距儀
區間測速系統
違停偵測系統
活動地磅系統
測速照相系統
AI 與人力資源分析 | 人才適任與久任度評估系統

專長領域：
人工智慧與機器學習 | 機器學習、生成式 AI、大語言模型 (LLM)
電腦視覺與影像處理 | 影像識別、智慧監控系統開發
嵌入式系統與數位電路 | 數位電路設計、嵌入式系統開發
軟體開發與工程 | 程式設計、多平台軟硬體整合

Question = “ 陳仁政的履歷 ? ”

prompt =
CV +
“\n 根據以上的資料請回答下面問題： \n” +
Question

根據以上的資料請回答下面問題：

陳仁政的履歷？

就算使用比較早期的模型，也可以正確的回答



Assistant `meta-llama-3.1-8b-instruct`

根據上面的資訊，我們可以知道以下的陳仁政的履歷：

1. 學術背景：博士學位在中原大學電子研究所
2. 研究經驗：博士後研究在中央研究院資訊科學研究所
3. 產業與教育經歷：
 - 資深工程師吉鴻電子
 - 正工程師冠捷科技
 - 資料科學家104人力銀行人資學院
 - 兼任實務教師長庚大學工商管理系
 - 顧問104人力銀行人資學院
4. 現職：

陳仁政的履歷？

即使使用超小型模型，回答的效果也是不錯



Assistant `gemma-3-1b-it`

好的，根據陳仁政提供的個人簡歷，以下是他的履歷重點：

核心能力 & 經驗：

- 人工智能產業協會講師 & 實踐大學推廣中心講師 & 緯育TibaMe顧問: 代表AI產業發展進行指導和培訓，涵蓋人工智慧、機器學習、數據分析等領域。
- 資深工程師/正工程師/資料科學家: 擁有豐富的工程經驗，參與了多項實際項目。
- 專案經驗:
 - 智慧金融 (收鈔機軟體開發): 參與開發偽鈔偵測技術。
 - 消費電子 (電視軟體開發): 專注於電視軟體開發。
 - 智慧交通與監控: 涉及智能交通和監控系統的開發，例如高承載管制違規偵測系統。
 - 手持式雷射測距儀, 區間測速系統, 違停偵測系統: 參與了多種實體監控技術的設計、開發和應用。
 - AI與人力資源分析 (人才適任與久任度評估系統): 利用人工智慧來輔助人力資源管理，提高人才選拔效率。

訊息 + 問題 = 提示詞 的缺點

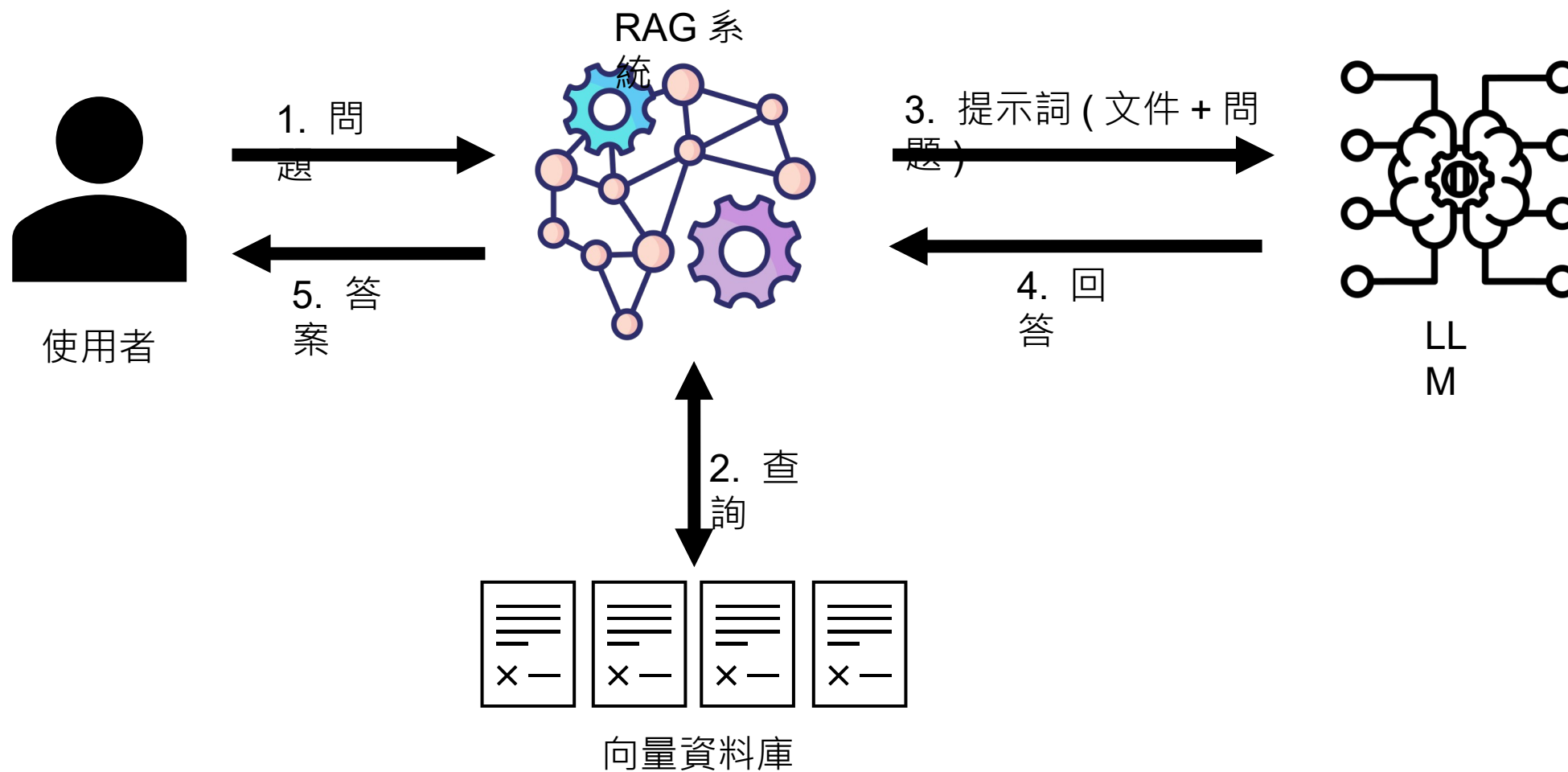
- 訊息會占用提示詞的輸入 tokens 空間
- 模型有最大 tokens 數量限制
- 訊息越長，模型推理越容易出錯

Gemini API 價格	Free Tier	Paid Tier, per 1M tokens in USD
Input price	Free of charge, use "gemini-2.5-pro-exp-03-25"	\$1.25, prompts <= 200k tokens \$2.50, prompts > 200k tokens
Output price (including thinking tokens)	Free of charge, use "gemini-2.5-pro-exp-03-25"	\$10.00, prompts <= 200k tokens \$15.00, prompts > 200k
Context caching price	Not available	Not available
Grounding with Google Search	Free of charge, up to 500 RPD	1,500 RPD (free), then \$35 / 1,000 requests
Used to improve our products	Yes	No

解決方法 - RAG

- 將文件拆分成多個小文件
- 僅將 **與問題相關的內容** 納入提示詞
- RAG 需要的組件
 - **多格式文件讀取器**：支援 PDF、Word、TXT 等格式讀入
 - **文件自動拆分器**：根據段落、語義或長度進行拆分
 - **文件向量化 (Embedding)**：將文字轉換為可比對的向量格式
 - **向量資料庫**：儲存並快速檢索向量資訊
 - **LLM 模型**：回答使用者問題，並與檢索內容整合
 - **RAG 整合系統**：結合以上模組，完成問答流程自動化

RAG 架構示意

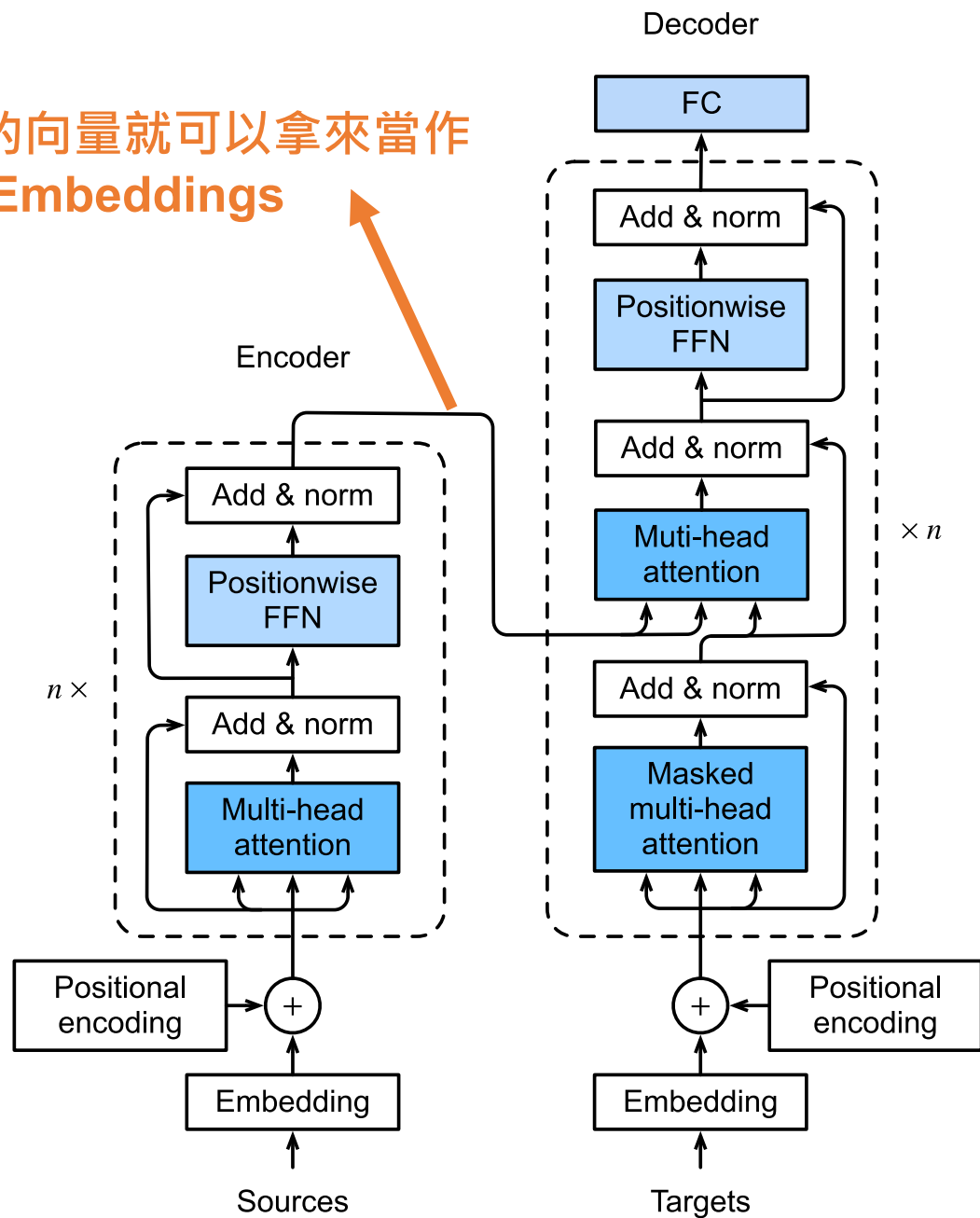


Text Embedding

如同 Word Vector，使用一組向量（多欄位數字）來編碼文字

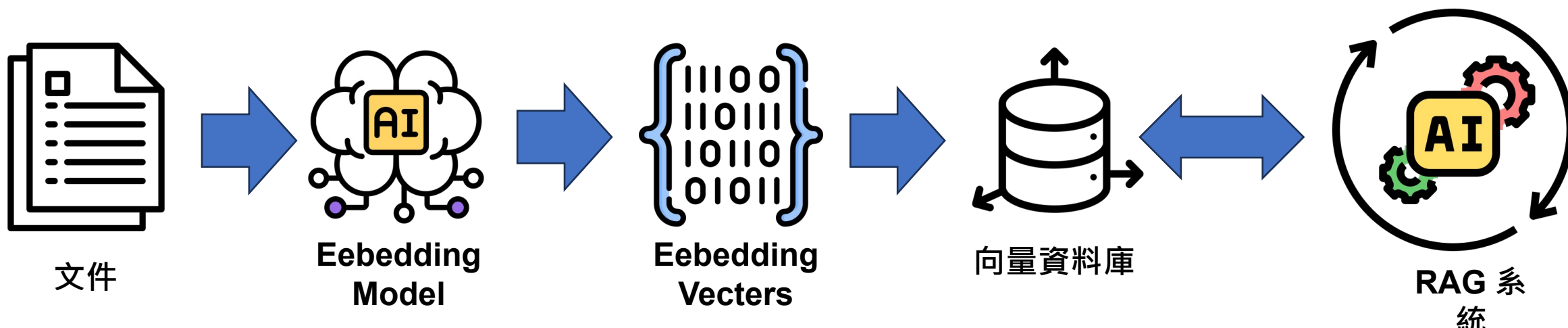
是否可以一樣用一組向量來編碼一串文字？

這裡的向量就可以拿來當作
Text Embeddings



向量資料庫

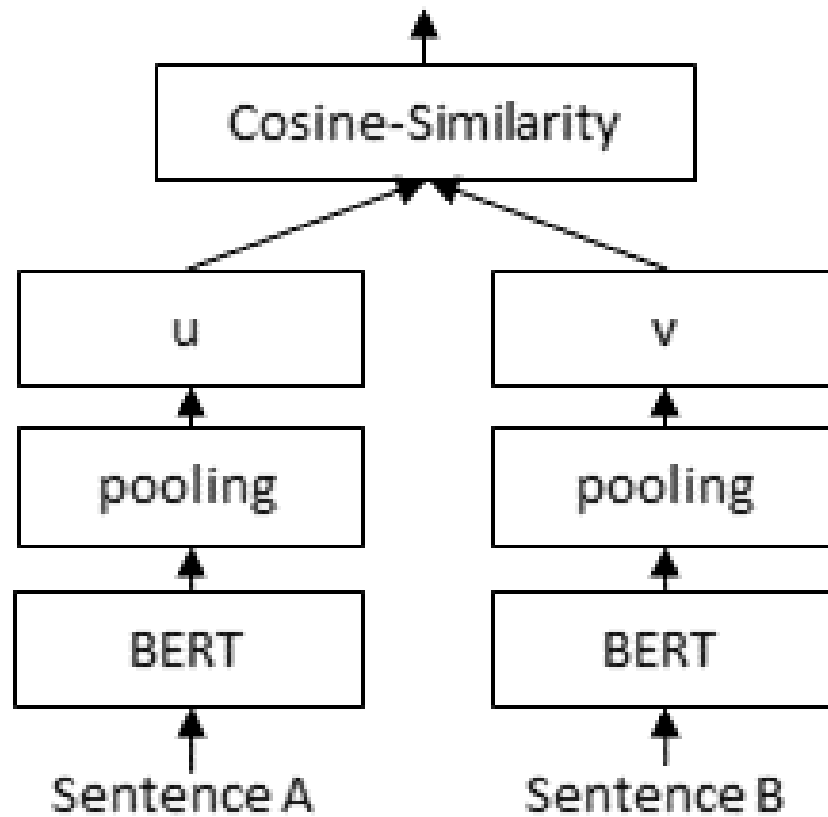
- 向量資料庫是一種專門設計用來儲存、索引和檢索向量 (vector) 資料的資料庫。
- 傳統關聯式資料庫擅長處理結構化資料。可是**相似度檢索任務**，表現就沒那麼理想。
- 向量資料庫則專門用來處理這類**高維空間中的相似度查詢** (如 Nearest Neighbor Search)



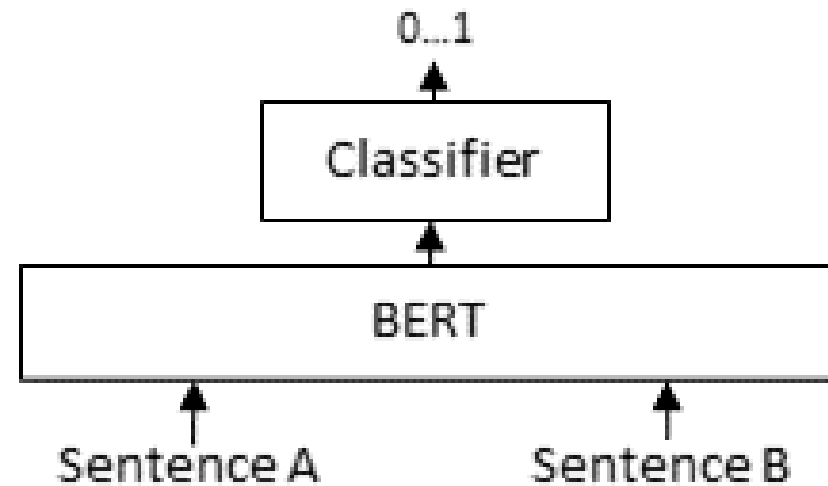
	Dedicated vector databases	Databases that support vector search
Open source (Apache 2.0 or MIT license)	 chroma  marqo  vespa  drant  LanceDB  Milvus	 OpenSearch  ClickHouse  PostgreSQL  cassandra
Source available or commercial	 Weaviate  Pinecone	 elasticsearch  redis  [ROCKSET]  SingleStore

Reranker

Bi-Encoder



Cross-Encoder



本機

應用程式行程 (Python)

Langchain

Vector Store

Embedding

Chat Model

Hugging Face

Chroma

SQL DB

Ollama

你的系統邏輯

LangChain

LLM Framework

LangChain

- LangChain 是一個**開源框架**
- 目的是簡化使用**大型語言模型 (LLM)** 開發應用程式的過程
- 提供了一系列的**工具、組件和抽象化**
- <https://www.langchain.com/>



- **鏈 (Chains)**

- LangChain 的基本構建區塊，代表著一系列串聯的操作。一個鏈可以包含多個組件，例如提示模板、LLM、輸出解析器等，用於執行特定的任務。

- **組件 (Components)**

- **提示 (Prompts)**：用於結構化 LLM 的輸入。
- **模型 (Models)**：與不同的 LLM (例如 ChatGPT, Gemini, 開源模型等) 進行互動的接口。
- **記憶 (Memory)**：用於在對話中保持上下文。
- **檢索 (Retrieval)**：用於從外部資料來源 (例如向量資料庫、文件等) 檢索相關資訊。
- **輸出解析器 (Output Parsers)**：用於結構化 LLM 的輸出。
- **代理 (Agents)**：能夠根據高階指令和可用的工具自主決策並採取行動的系統。
- **工具 (Tools)**：代理可以調用的外部功能，例如搜尋引擎、資料庫查詢工具等。

Langchain 優點

- 模組化設計，拼積木式開發
- 高度抽象化，專注業務邏輯
- 多模型、多工具整合
- 社群大，資源豐富

Langchain 缺點

- 版本更新快但缺乏向下相容性
- 功能封裝太深，不易理解內部邏輯
- 為了支援多模型，犧牲了最佳化空間
- 學習曲線陡峭
- 抽象層級太多， debug 困難

FineTune

大型語言模型訓練

- Pretrain （預訓練）
 - 學習語言的一般結構和知識
 - 使用大量通用文本（如維基百科、書籍、新聞等）來訓練模型預測下一個單詞或填空詞
 - 通常採用自監督學習（ self-supervised learning ）
- DAPT （ Domain-Adaptive Pretraining ，領域適應預訓練）
 - 讓模型熟悉特定領域的語言風格和知識
 - 在已經預訓練的模型基礎上，再用特定領域的文本進行額外預訓練
 - 模型能更好地理解 and 生成特定領域的專業語言和概念

大型語言模型訓練

- SFT （ Supervised Fine-Tuning ， 監督微調）
 - 讓模型學會特定任務
 - 使用人工標註的「輸入 → 輸出」資料對模型微調
 - 模型能直接做任務，如聊天、摘要、分類
- RLHF （ Reinforcement Learning from Human Feedback ， 人類反饋強化學習）
 - 用人類反饋引導模型生成「更好」的回答
 - 收集人類偏好或評分（人類對模型回答的好壞打分）
 - 訓練一個 reward model （獎勵模型）
 - 用強化學習（RL）調整生成模型，使其最大化人類偏好分數
 - 模型能生成「符合人類期望」的回答，減少不當或荒謬輸出

大型語言模型訓練

- DPO （ Direct Preference Optimization ，直接偏好優化）
 - 直接從偏好標註資料中學習優化生成結果
 - 收集人類對生成結果的偏好（哪個回答更好），直接訓練模型生成「更被人類喜歡」的輸出
 - 與傳統 RLHF 相似，但不需要額外的策略梯度或 reward 模型訓練，而是直接利用偏好資料進行優化
 - 生成結果更符合人類偏好，例如回答更精準、禮貌或簡潔

LoRA

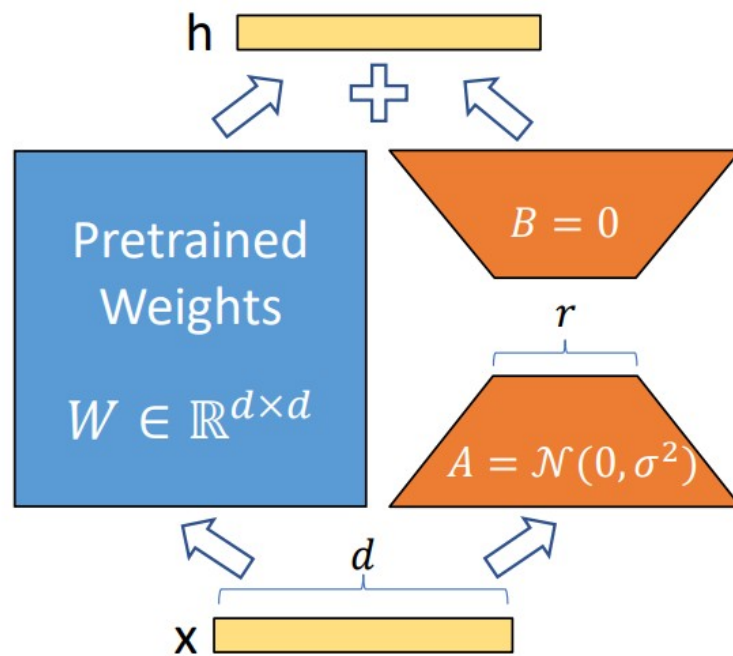


Figure 1: Our reparametrization. We only train A and B .

VRAM need for LLM model fine-tuning

Method	Precision	7B	13B	30B	70B	110B
Full	16	67GB	125GB	288GB	672GB	1056GB
LoRA	16	15GB	28GB	63GB	146GB	229GB
QLoRA	8	9GB	17GB	38GB	88GB	138GB
QLoRA	4	5GB	9GB	20GB	46GB	72GB

FineTune 的資料格式

- 在 fine-tune 一個大型語言模型 (LLM) 時，常見的資料格式會有三個欄位：**instruction**, **input**, **output**。這種格式常用於「指令微調」(instruction tuning)，讓模型學會根據指令來產生對應的回答。
- 欄位的意義如下：
 - **instruction**：給模型的一個明確任務指令，說明你想讓它做什麼。
 - **input**：指令的具體內容或上下文資料（可選，有些任務不需要 input）。
 - **output**：期望模型根據 instruction 和 input 所產生的回答或結果。

FineTune 的資料範例 (1/2)

問答任務 (Question Answering)

```
{  
  "instruction": " 請根據下列段落回答問題。 ",  
  "input": " 段落：貓是夜行性動物，牠們通常在夜間活動。問題：貓什麼時候最活躍？ ",  
  "output": " 貓在夜間最活躍。 "  
}
```

翻譯任務 (Translation)

```
{  
  "instruction": " 請將下列英文翻譯成中文。 ",  
  "input": "The weather is nice today.",  
  "output": " 今天天氣很好。 "  
}
```


FineTune 的資料範例 (2/2)

寫作輔助 (Text Generation)

```
{  
  "instruction": " 寫一篇關於人工智慧未來發展的短文。",  
  "input": "",  
  "output": " 人工智慧（AI）在未來將扮演越來越重要的角色，從自駕車到醫療診斷 ..."  
}
```

數學解題 (Math Problem Solving)

```
{  
  "instruction": " 請解釋並解出以下數學問題。",  
  "input": " 如果一個蘋果 3 元，一個香蕉 2 元，我有 11 元，最多可以買幾個水果？",  
  "output": " 最多可以買 5 個水果（3 個蘋果和 2 個香蕉）。"  
}
```

PEFT

- 參數高效率微調 Parameter-Efficient Fine-Tuning
- Hugging Face 的套件
 - <https://github.com/huggingface/peft>
- 大型預訓練模型進行微調通常成本高
- PEFT 只調整模型的一小部分參數
- 降低微調成本，同時仍保持良好的效能

PEFT 的資料格式轉換步驟

課程測試用資料

```
{  
  "instruction": " 陳仁政是哪年出生的？ ",  
  "input": "",  
  "output": " 他是 1972 年出生 "  
}
```

第一步

將訓練資料組合成一個完整的提示詞，包含輸入問句跟輸出

prompt = instruction + output

= “ 陳仁政是哪年出生的？他是 1972 年出生”

PEFT 的資料格式轉換步驟

`prompt = “ 陳仁政是哪年出生的？他是 1972 年出生”`

第二步

使用 `AutoTokenizer` 將上面的 `prompt` 進行編碼

產生三個變數

`input_ids : prompt 編碼`

`'input_ids': tensor([128000, 118741, 109639, 75513, 21043, 106189, 8107, 20834, 122714, 11571, 43511, 21043, 4468, 17, 8107, 20834, 21990])`

`attention_mask : 全部都是 1 的陣列，長度跟 input_ids 一樣`

`'attention_mask': tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])`

`labels : prompt 的編碼，但是前面問句的部分全部為 0`

`'labels': tensor([ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 43511, 21043, 4468, 17, 8107, 20834, 21990])`

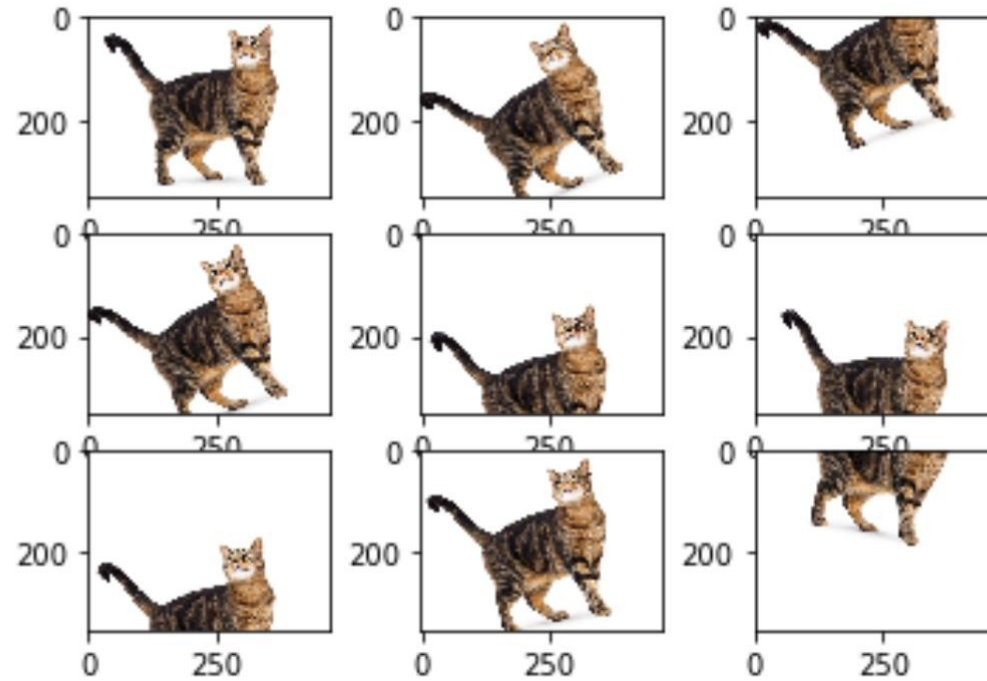
資料擴增 Data Augmentation

- 一種用來增加訓練資料多樣性的技術
- 應用在機器學習，特別是深度學習模型的訓練過程中
- 透過對現有資料的修改或變換，生成新的資料樣本
- 提升模型的泛化能力，避免 overfitting（過擬合）

圖像上的常見資料擴增技術

技術	說明
翻轉 (Flip)	垂直或水平翻轉圖片
旋轉 (Rotate)	對圖片進行任意角度的旋轉
裁切 (Crop)	隨機裁剪部分圖像
縮放 (Zoom)	對圖片放大或縮小
平移 (Shift)	向某方向平移圖片
加噪聲 (Noise)	加入隨機雜訊模擬不同情況
顏色變換	調整亮度、對比、飽和度等

Image Data Augmentation



NLP 的資料擴增方式

技術	說明
同義詞替換	將詞句中的單字替換成同義詞
隨機刪除 (Random Deletion)	隨機刪除一些字詞
隨機插入 (Random Insertion)	插入與上下文有關的字詞
反轉 (Back Translation)	翻譯成另一語言再翻譯回來

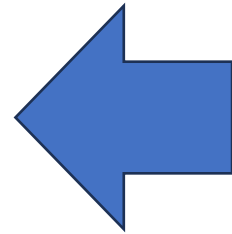
大語言模型微調 - 關於訓練資料

- 資料數量要多
- 資料要具備多樣性
 - 相同的問題，可以採用各種不同的問句方式增加多樣性

要如何退貨？



- 請問我想退貨要怎麼辦理？
- 我買的東西不合用，可以退貨嗎？
- 寄來的東西不能用，我要退貨並退款
-
-
-



讓 LLM 來幫你
進行資料擴增

大型語言模型微調的缺點

- 需要重新訓練來更新知識，較難實現動態更新。
- 訓練成本和時間高，尤其對大模型。
- 泛化能力較差，容易過擬合。
- 無法靈活應對不在訓練資料中的新知識或場景。

LLamaFactory

- <https://github.com/hiyouga/LLamaFactory>
- 大型語言模型（LLM）微調與訓練框架
- 提供了一套完整的工具鏈，用來微調、後訓練、評估與部署 LLMs
- 訓練方式
 - 全參數微調
 - 凍結微調（Freeze Tuning）
 - 參數效率訓練（如 LoRA / QLoRA / AdaLoRA 等）
 - 增量預訓練
 - 討論式微調（PPO/DPO/KTO/ORPO 等）

Tools / Function Call

Tools / Function call

- 什麼是 Function Call ?
 - 讓大型語言模型（LLM）呼叫外部工具或 API 的能力。
 - 模型可依照指令結構，自動選擇要使用的功能並傳入參數。
- 常見用途
 - 查天氣
 - 查股票、匯率資訊
 - 啟動計算工具或資料查詢系統
 - 自動撰寫程式碼並執行

Function Call 概念示範

為了提升任務表現，我們可於提示詞中詳列 LLM 可使用之外部功能，並明確要求其在執行任務時適時加以應用。

```
System_Prompt = """
```

```
你是一個書店購物 AI 助手。你的任務是：
```

```
1. 根據用戶提供的書名和購買數量
```

```
2. 回傳 add_to_cart( 書名 , 數量 )， 不要任何額外的解釋或判斷， 僅需回傳函式呼叫。
```

```
3. 當資訊不完整時， 需要主動詢問缺失的商品名稱和購買數量
```

```
"""
```

使用 Gemini 的測試效果：

你好

```
System_Prompt = """
```

你是一個書店購物 AI 助手。你的任務是：

1. 根據用戶提供的書名和購買數量
2. 回傳 `add_to_cart(書名, 數量)`，不要任何額外的解釋或判斷，僅需回傳函式呼叫。
3. 當資訊不完整時，需要主動詢問缺失的商品名稱和購買數量

```
"""
```

缺點：

1. 提示詞設計具有一定挑戰性，往往需要經過多次調整與優化，才能達到預期效果。
2. 缺乏標準化機制來準確描述函數的功能與參數，需透過反覆測試與驗證，才能確保語言模型正確理解與回應。

書

書籍訂購範例展示 · 自訂 Gem

您好，請問您想訂購哪本書？需要幾本呢？

👍 🗨️ ↩️ ⋮

我想要買實戰PYTHON

書

書籍訂購範例展示 · 自訂 Gem

請問您需要幾本呢？

👍 🗨️ ↩️ ⋮

我需要三本，有貨嗎？

書

書籍訂購範例展示 · 自訂 Gem

`add_to_cart("實戰PYTHON", 3)`

👍 🗨️ ↩️ ⋮

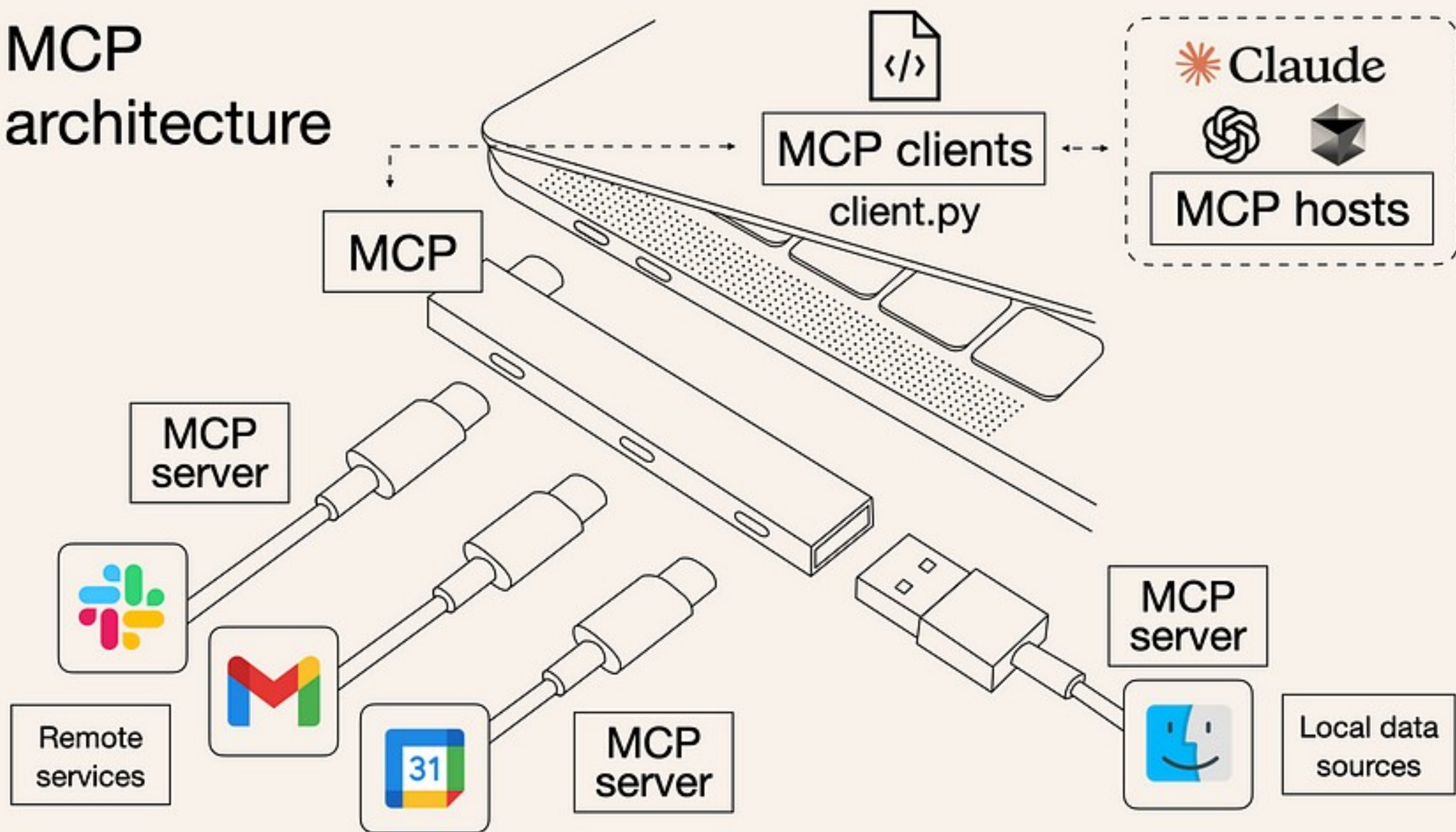


系統可監控大型語言模型 (LLM) 的回應內容，並在偵測到符合函數格式的指令時，自動進行解析與執行。

MCP



MCP architecture



本機

應用程式行程 (Python)

MCP SDK

Langchain

你的系統邏輯
(MCP Client)

Chat Model

Hugging Face

MCP Server

stdio

http

MCP
Server

Ollama

Ollama

MCP Server 測試

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "C://DATA"
      ]
    }
  }
}
```

測試平台 (Client) : Cherry Studio, Gemini Code, Claude Desktop



END

